ĐẠI HỌC QUỐC GIA HÀ NỘI

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



Nguyễn Ngọc Hải

ROBOT DI ĐỘNG NHẬN DẠNG VÀ CẢNH BÁO SỚM ĐÁM CHÁY SỬ DỤNG TRÍ TUỆ NHÂN TẠO

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Công nghệ kỹ thuật Cơ điện tử

HÀ NỘI - 2023

ĐẠI HỌC QUỐC GIA HÀ NỘI

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

Nguyễn Ngọc Hải

ROBOT DI ĐỘNG NHẬN DẠNG VÀ CẢNH BÁO SỚM ĐÁM CHÁY SỬ DỤNG TRÍ TUỆ NHÂN TẠO

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Công nghệ kỹ thuật Cơ điện tử

Cán bộ hướng dẫn: TS. Phạm Văn Bạch Ngọc

(Ký tên)

HÀ NỘI – 2023

ROBOT DI ĐỘNG NHẬN DẠNG VÀ CẢNH BÁO SỚM ĐÁM CHÁY SỬ DỤNG TRÍ TUỆ NHÂN TẠO

Nguyễn Ngọc Hải

Khóa QH-2019-I/CQ-CLC, ngành Công nghệ kỹ thuật Cơ điện tử

Tóm tắt khóa luận tốt nghiệp

Việc ứng dụng các loại robot khác nhau nhằm phục vụ và thay thế con người trong việc giám sát và cảnh báo rủi ro ngày càng thu hút sự quan tâm nghiên cứu của nhiều nước trên thế giới, đặc biệt là việc cảnh báo nguy cơ cháy nổ. Các tai nạn do cháy nổ gây ra ra có thể gây thiệt hại rất lớn và nguy hiểm cho con người, vì vậy, việc giám sát và cảnh báo đám cháy giúp giảm thiểu thiệt hại do cháy nổ gây ra. Trong đề tài này em sẽ trình bày mô hình robot di động nhận dạng và cảnh báo sớm đám cháy sử dụng trí tuệ nhân tạo giúp giám sát và phát hiện, đưa ra cảnh báo sớm nguy cơ cháy nổ. Robot được hoạt thiết kế với mô hình 3 bánh Omni, với các trục bánh xe lệch với nhau góc 120 độ với nhiệm vụ di chuyển linh hoạt. Một máy tính nhúng sẽ được sử dụng cho nhiệm vụ giám sát thông tin hình ảnh tại khu vực cần giám sát, nếu có nguy cơ cháy nổ như lửa và khói thì thông tin cảnh báo sẽ được gửi đi. Vi điều khiển và mạch cầu sẽ giúp điều khiển động cơ DC trên robot, ngoài ra vi điều khiển cũng sẽ giao tiếp với máy tính nhúng để có thể di chuyển đến tọa độ đang có nguy cơ cháy để gia tăng độ chính xác trong nhận dạng. Động cơ DC có tích hợp encoder giúp xác định vận tốc, quãng đường di chuyển của từng bánh xe, từ đó xác định được tọa độ của robot. Các kết quả về nghiên cứu mô hình, điều khiển chuyển động với bánh xe Omni, nhận dạng nguy cơ cháy và cảnh báo của robot sẽ được em trình bày trong khóa luận này.

Từ khóa: *Robot giám sát và cảnh báo cháy, điều khiển chuyển động của Robot, bánh xe Omni, nhận dạng nguy cơ cháy, ...*

LỜI CAM ĐOAN

Tôi xin cam đoan khóa luận tốt nghiệp "Robot di động nhận dạng và cảnh báo sớm đám cháy sử dụng trí tuệ nhân tạo" là công trình nghiên cứu của tôi dưới sự hướng dẫn của TS. Phạm Văn Bạch Ngọc. Những phần sử dụng tài liệu tham khảo trong khóa luận đã được nêu rõ trong phần tài liệu tham khảo, các số liệu, kết quả trình bày trong khóa luận là hoàn toàn trung thực, nếu sai tôi xin hoàn toàn chịu trách nhiệm và chịu mọi kỷ luật của khoa và nhà trường đề ra.

Hà Nội, ngày 24 tháng 05, 2023

Sinh viên

Nguyễn Ngọc Hải

LỜI CẢM ƠN

Được sự phân công của khoa Cơ học kỹ thuật và Tự động hóa, Trường Đại Học Công Nghệ - Đại học Quốc Gia Hà Nội, và sự đồng ý của cán bộ hướng dẫn – TS. Phạm Văn Bạch Ngọc, em đã thực hiện đề tài: "Robot di động nhận dạng và cảnh báo sớm đám cháy sử dụng trí tuệ nhân tạo".

Để hoàn thành được khóa luận này, em xin được gửi lời cảm ơn chân thành đến Ban giám hiệu nhà trường, các phòng ban, quý thầy cô giáo khoa Cơ học kỹ thuật và Tự động hóa - Trường Đại Học Công Nghệ - Đại Học Quốc gia Hà Nội đã giảng dạy và tạo mọi điều kiện cho em học tập và hoàn thành khóa luận tốt nghiệp.

Đặc biệt, bằng lòng kính trọng và biết ơn sâu sắc, em xin chân thành cảm ơn TS. Phạm Văn Bạch Ngọc - người luôn tận tình hướng dẫn chỉ bảo, chia sẻ và động viên em vượt qua những khó khăn để em hoàn thành khóa luận tốt nghiệp này. Cảm ơn thầy đã giúp em cùng rất nhiều các bạn khác có thể tự tin vững bước hơn trên con đường phía trước.

Cuối cùng em xin gửi lời cảm ơn đến gia đình và bạn bè đã hết lòng giúp đỡ, khích lệ, động viên em trong suốt quá trình em học tập và nghiên cứu.

Mặc dù bản thân đã có nhiều cố gắng để có thể thực hiện đề tài một cách hoàn chỉnh nhất, song do hạn chế về kiến thức và kinh nghiệm nên không thể nào tránh được những thiếu sót nhất định mà bản thân không nhận ra được. Em rất mong nhận được sự chỉ bảo của thầy, cô giáo cùng các bạn để khóa luận có thể hoàn chỉnh hơn.

Em xin chân thành cảm ơn!

Hà Nội, ngày 24 tháng 05, 2023

Sinh viên

Nguyễn Ngọc Hải

MỤC LỤC

MỞ ĐẦU *1*

CHƯƠNG 1: TỔNG QUAN *4*

1.1. Các mẫu robot nhận dạng cháy hiện nay *4*

1.1.1. Robot nhận dạng cháy sử dụng đa cảm biến kết hợp *4*

1.1.2. Robot nhận dạng cháy và điều khiển hướng di chuyển *5*

1.2. Các phương án lựa chọn mô hình bánh xe và điều khiển Robot *7*

1.2.1. Thiết kế *8*

1.2.2. Mô hình *9*

1.2.3. Tính ổn định *10*

1.2.4. Khả năng cơ động *13*

1.2.5. Khả năng điều khiển *14*

1.3. Mục tiêu và đối tượng nghiên cứu *14*

1.4. Nhiệm vụ của đề tài và phạm vi nghiên cứu *15*

1.4.1. Nhiệm vụ của đề tài *15*

1.4.2. Phạm vi nghiên cứu *15*

1.4.3. Phương pháp nghiên cứu *15*

1.5. Kết luận chương *15*

CHƯƠNG 2: PHÂN TÍCH ĐỘNG HỌC VÀ ĐỘNG LỰC HỌC ROBOT 3 BÁNH OMNI *17*

2.1. Phân tích bánh xe Omni *17*

2.2. Xác định vị trí Robot *18*

2.3. Phân tích động học robot 3 bánh Omni *21*

2.4. Phân tích động lực học robot 3 bánh Omni *23*

CHƯƠNG 3: MÔ HÌNH 3D, THUẬT TOÁN ĐIỀU KHIỂN VÀ NHẬN DẠNG CỦA ROBOT *25*

3.1. Xây dựng mô hình 3D cho robot *25*

3.2. Thuật toán điều khiển robot 27

3.2.1. Điều khiển Robot tự động di chuyển trong theo tọa độ khi chưa nhận dạng được nguy cơ cháy 27

3.2.2. Điều khiển robot tự động di chuyển lại gần nguy cơ cháy khi phát hiện có nguy cơ cháy 28

3.3. Mô hình nhận dạng nguy cơ cháy 31

3.3.1. Mô hình Yolov5 trong nhận dạng đám cháy 31

3.3.2. Chuẩn bị và tiền xử lý dữ liệu 35

3.3.3. Thuật toán xác định vị trí sau khi phát hiện nguy cơ cháy 37

CHƯƠNG 4: ĐIỀU KHIỂN VÀ THỰC NGHIỆM 40

4.1. Điều khiển robot sử dụng 3 bánh Omni 40

4.1.1. Thu thập tín hiệu encoder để và sử bộ điều khiển PID để tính toán công suất cho động cơ 40

4.1.2. Điều khiển động cơ bằng phương pháp điều chế độ rộng xung 44

4.2. Thực nghiệm với robot sử dụng 3 bánh Omni 47

KẾT LUẬN 53

PHỤ LỤC 54

TÀI LIỆU THAM KHẢO 65

Danh mục hình ảnh

Hình 1. 1 Robot nhận dạng cháy sử dụng cảm biến kết hợp.....4

Hình 1. 2 Sơ đồ quy trình phát hiện cháy.....5

Hình 1. 3 Sơ đồ khối của Robot nhận dạng cháy và điều khiển hướng di chuyển.....6

Hình 1. 4 Sơ đồ hoạt động của robot nhận dạng lửa và điều khiển không dây.....6

Hình 1. 5 Hình ảnh robot phát hiện cháy và điều khiển hướng di chuyển.....7

Hình 1. 6 Mô hình bánh xe cơ bản có thể lựa chọn cho robot nhận dạng và cảnh báo sớm nguy cơ cháy. 8

Hình 1. 7 Robot Carnegie Mellon Uranus, sử dụng 4 bánh xe Mecanum 14

Hình 2. 1 Hai hướng di chuyển của bánh xe Omni 90 độ.....17

Hình 2. 2 Robot 3 bánh sử dụng bánh xe Omni [4] 18

Hình 2. 3 Robot 4 bánh sử dụng bánh xe Omni [5] 18

Hình 2. 4 Hệ trục tọa độ tổng thể và hệ trục tọa độ cơ sở của Robot.....19

Hình 2. 5 Robot di động được căn chỉnh theo hệ trục tọa độ tổng thể. 20

Hình 2. 6 Một robot dẫn động vi sai trong hệ tọa độ tổng thể.....21

Hình 2. 7 Bài toán động học cho robot 3 bánh Omni 22

Hình 3. 1 Mô hình 3D robot 3 bánh Omni.....25

Hình 3. 2 Mô hình tổng thể của Robot khi nhìn ngang 26

Hình 3. 3 Mô hình bánh xe Omni 26

Hình 3. 4 Quỹ đạo di chuyển của robot khi không phát hiện nguy cơ cháy 27

Hình 3. 5 Robot khi phát hiện và tính toán vị trí nguy cơ cháy 28

Hình 3. 6 Quỹ đạo di chuyển của robot khi nhận dạng được nguy cơ cháy 29

Hình 3. 7 Ví dụ về chia lưới và vẽ đường bao cho vật thể 32

Hình 3. 8 Anchor Box cho một object.....35

Hình 3. 9 Gắn nhãn và xuất đặc trưng theo định dạng cua YOLO.....37

Hình 3. 10 Tệp cấu hình của bộ dữ liệu. 37
Hình 3. 11 Hệ thống camera kép sử dụng trong tính toán khoảng cách.....38
Hình 3. 12 Các góc của máy ảnh 39
Hình 4. 1 Cấu tạo của encoder.....40
Hình 4. 2 Tín hiệu của encoder khi hoạt động 41
Hình 4. 3 Tín hiệu encoder khi motor quay cùng và ngược chiều kim đồng hồ 42
Hình 4. 4 Xung A và B lệch pha nhau 90 độ 43
Hình 4. 5 Lập trình bộ điều khiển PID tính toán độ rộng xung đầu ra cho mỗi motor 44
Hình 4. 6 Phương pháp điều chế độ rộng xung 45
Hình 4. 7 Mạch cầu H sử dụng MOSFET 45
Hình 4. 8 Sơ đồ kết nối giữa vi điều khiển, mạch cầu H và 2 động cơ DC.....46
Hình 4. 9 Lập trình tính toán và điều khiển động cơ dựa trên bộ điều khiển PID 47
Hình 4. 10 Robot chạy thực nghiệm 47
Hình 4. 11 Kết quả huấn luyện mô hình.....48
Hình 4. 12 Kết quả thử nghiệm mô hình nhận dạng 49
Hình 4. 13 Kết quả thử nghiệm mô hình nhận dạng 49
Hình 4. 14 Sơ đồ khối của mô hình Robot.....50
Hình 4. 15 Thực nghiệm nhận dạng và cảnh báo nguy cơ cháy trên robot 51
Hình 4. 16 Robot thực hiện điều hướng theo hướng của nguy cơ cháy.....51

MỞ ĐẦU

Tính cấp thiết của đề tài

Ngành công nghiệp robot thực sự bùng nổ trong những thập kỷ gần đây, các công nghệ mới đã và đang được thử nghiệm tại các cơ quan, tổ chức và các trường đại học khác nhau trên toàn thế giới. Trong đó, ứng dụng robot thay thế con người trong những công việc yêu cầu tính giám sát cao đang trở thành một xu hướng phát triển và là một trong các lĩnh vực nghiên cứu quan trọng vì hệ thống này có thể hoàn thành nhiệm vụ thay cho con người với sự đảm bảo về năng suất và chỉ yêu cầu năng lượng để hoạt động.

Robot di động có nhiệm vụ nhận dạng và cảnh báo sớm nguy cơ cháy là robot có thể di chuyển linh hoạt trong một khu vực với nhiệm vụ giám sát và phát hiện nguy cơ cháy nổ, từ đó đưa ra cảnh báo để có thể có những phương án xử lý từ sớm, hạn chế rủi ro và thiệt hại do cháy nổ gây ra. Với khả năng thực hiện giám sát trong thời gian dài và chỉ yêu cầu năng lượng để cung cấp cho robot hoạt động, robot di động nhận dạng và cảnh báo sớm nguy cơ cháy nổ không chỉ giúp nâng cao hiệu quả giám sát, cảnh báo sớm với các nguy cơ cháy nổ mà còn giúp tiết kiệm sức lực lao động từ con người mà vẫn đảm bảo được yêu cầu công việc.

Ý nghĩa khoa học, thực tiễn

Robot di động nhận dạng và cảnh báo sớm đám cháy là một phần của quá trình phát triển nền công nghiệp hiện đại ứng dụng công nghệ vào thay thế nhân lực và là bước đà phát triển trong ngành chế tạo robot.

Kết quả trong khóa luận này là robot di động nhận dạng và cảnh báo sớm đám cháy có thể thay thế nhiệm vụ giám sát và cảnh báo cháy của con người, đảm bảo việc giảm thiểu tối đa thiệt hại do các vụ tại nạn cháy nổ có thể gây ra.

Đối tượng và phương pháp nghiên cứu

Đề tài bao gồm những đối tượng nghiên cứu sau đây:

Phần mềm lập trình Arduino IDE.

Phần mềm lập trình Pycharm.

Phầm mềm thiết kế 2D AutoCad.

Phần mềm thiết kế 3D Solidworks.

Máy tính nhúng Jetson Nano Develop Kit.

Vi điều khiển Arduino Mega 2560.

Mạch cầu H.

Động cơ DC Servo JGB37-520 DC Geared Motor.

Để hoàn thành được khóa luận, việc tìm hiểu cụ thể từng đối tượng và vận dụng nguyên lý hoạt động sao cho phù hợp là phương pháp nghiên cứu hiệu quả.

Nội dung nghiên cứu

Nội dung chính của đề tài chế tạo và điều khiển mô hình robot di động sử dụng trí tuệ nhân tạo để nhận dạng và chẩn đoán sớm đám cháy. Trong chế tạo, robot cần có 3 trục chính của bánh xe tạo với nhau một góc 120 độ. Robot cần di chuyển linh hoạt dựa trên bài toán động học với bánh xe Omni và động cơ DC tích hợp encoder, đảm bảo sai số nhỏ nhất và có những phương pháp triệt tiêu sai số để robot có thể hoạt động độc lập, không cần điều khiển. Đồng thời trong thời gian di chuyển, robot cần thu thập thông tin về hình ảnh từ camera, xử lý và đưa ra dự đoán về nguy cơ cháy. Nếu dự đoán được nguy cơ cháy, robot cần đưa ra cảnh báo.

CHƯƠNG 1: TỔNG QUAN

Tìm hiểu tổng quan khóa luận "Robot di động nhận dạng và cảnh báo sớm đám cháy sử dụng trí tuệ nhân tạo"

CHƯƠNG 2: PHÂN TÍCH ĐỘNG HỌC VÀ ĐỘNG LỰC HỌC ROBOT 3 BÁNH OMNI

Phân tích hệ thống tọa độ, tính chính xác, mô hình động học của robot di động nhận dạng và chẩn đoán sớm đám cháy.

CHƯƠNG 3: MÔ HÌNH 3D, THUẬT TOÁN ĐIỀU KHIỂN VÀ NHẬN DẠNG CỦA ROBOT

Xây dựng mô hình và thuật toán điều khiển di chuyển của robot trong hệ tọa độ không gian.

CHƯƠNG 4: ĐIỀU KHIỂN VÀ THỰC NGHIỆM

Điều khiển robot di chuyển theo bài toán động học và kết quả thực nghiệm

CHƯƠNG 5: TỔNG KẾT

CHƯƠNG 1: TỔNG QUAN

1.1. Các mẫu robot nhận dạng cháy hiện nay

1.1.1. Robot nhận dạng cháy sử dụng đa cảm biến kết hợp

Robot kết hợp các cảm biến nhận dạng lửa được Sucuoglu vcs [1] phát triển để dự đoán nguy cơ cháy nổ xảy ra, trong đó, cảm biến khói, cảm biến lửa và cảm biến nhiệt độ được kết hợp với nhau để có thể đưa ra dự đoán với độ tin cậy cao về nguy cơ cháy.

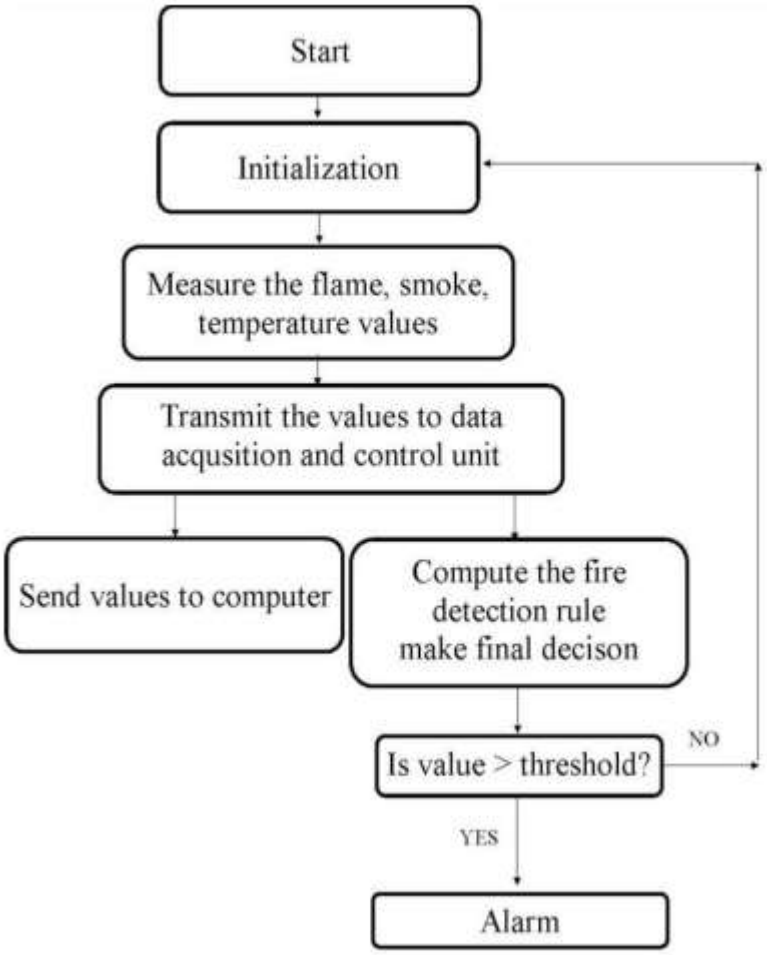


Hình 1. 1 Robot nhận dạng cháy sử dụng cảm biến kết hợp

Quá trình đánh giá nguy cơ cháy dựa trên ba loại cảm biến: cảm biến nhiệt độ, cảm biến khói và cảm biến lửa. Thuật toán dựa trên thông số đo được từ ba cảm biến với đầu vào là nhiệt độ (X_1), lửa (X_2) và khói (X_3). Các thông số đo được từ ba cảm biến được kết hợp cùng các trọng số và cho ra kết quả cuối cùng "T" với công thức như sau:

$$T = W_1X_1 + W_2X_2 + W_3X_3 \qquad (1.1)$$

Khi đó, nếu giá trị "T" vượt ngưỡng cho phép, robot sẽ cho đó là nguy cơ cháy.

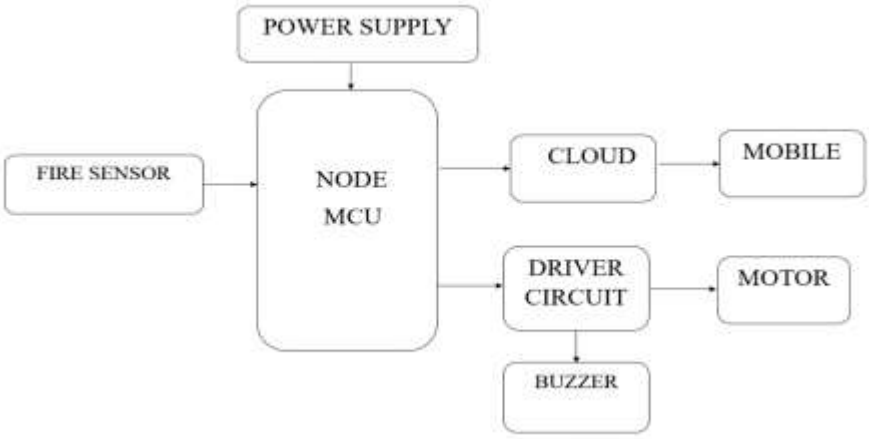


Hình 1. 2 Sơ đồ quy trình phát hiện cháy

Các thử nghiệm cho các khoảng cách phát hiện và nguồn lửa khác nhau để xác định giá trị trọng số tối ưu cho từng cảm biến và để có được thông tin phát hiện cháy chính xác. Để rút ra phương trình phát hiện cháy; phân tích PLS (Partial Least Square) đã được sử dụng. Từ đó rút ra được các trọng số cho nhiệt độ, lửa và khói lần lượt là: 0.16; 0.77 và 0.07. Do đó, ta có công thức tính "T":

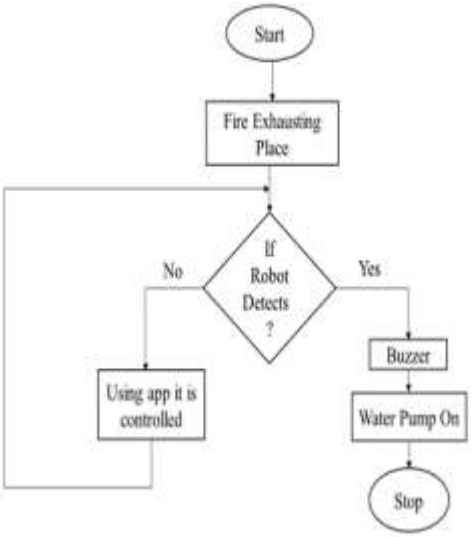
1.1.2. Robot nhận dạng cháy và điều khiển hướng di chuyển

Robot nhận dạng cháy và điều khiển hướng di chuyển được phát triển bởi Sridevi vcs [2] ứng dụng cảm biến lửa và các phương pháp điều khiển không dây để điều khiển Robot dập tắt đám cháy. Sơ đồ khối của Robot nhận dạng cháy và điều khiển hướng di chuyển như sau:



Hình 1. 3 Sơ đồ khối của Robot nhận dạng cháy và điều khiển hướng di chuyển

Robot được xây dựng từ các khối nguồn; khối cảm biến lửa; khối điều khiển dựa trên điện thoại và truyền thông tin qua đám mây; khối cảnh báo; driver và động cơ. Với các khối trên, robot có thể nhận dạng lửa và điều khiển dựa trên sơ đồ sau.



Hình 1. 4 Sơ đồ hoạt động của robot nhận dạng lửa và điều khiển không dây

Robot được điều khiển bởi các câu lệnh thông qua ứng dụng điện thoại, nếu như cảm biến lửa phát hiện được lửa, nước sẽ được bơm qua vòi thực hiện quy trình dập lửa, ngoài ra chuông cũng sẽ được kích hoạt để đưa ra cảnh báo về nguy cơ cháy.

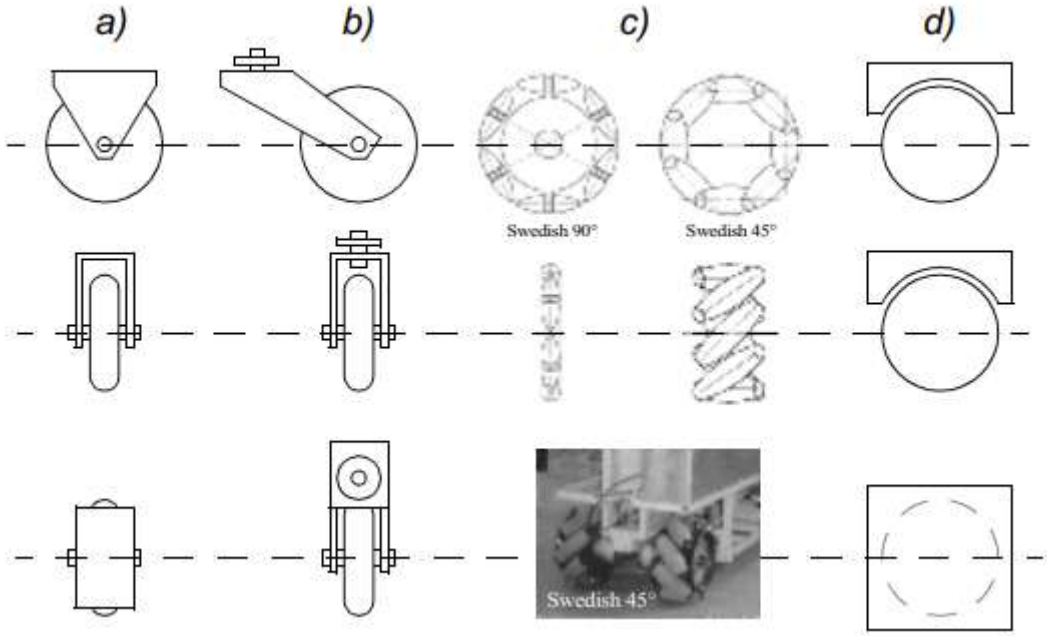


Hình 1. 5 Hình ảnh robot phát hiện cháy và điều khiển hướng di chuyển

Có thể thấy việc ứng dụng robot thay thế con người trong giám sát, phát hiện và cảnh báo nguy cơ cháy đang ngày càng trở thành một xu hướng phát triển của robot do rất nhiều ưu điểm nó mang lại so với những hệ thống phát hiện và cảnh báo cháy hiện nay, tuy nhiên việc hoàn thiện robot đảm bảo khả năng di chuyển, nhận dạng và cảnh báo cháy vẫn cần được hoàn thiện và phát triển hơn nữa dựa trên những tính năng hiện có để có thể linh hoạt và đưa ra được những cảnh báo sớm nhất đối với nguy cơ hỏa hoạn.

1.2. Các phương án lựa chọn mô hình bánh xe và điều khiển Robot

Bánh xe cho đến nay vẫn là cơ chế chuyển động phổ biến nhất trong robot di động và trong các phương tiện nhân tạo nói chung do chúng có cơ cấu đơn giản và đạt được hiệu quả tốt với yêu cầu di chuyển của robot. Ngoài ra việc cân bằng của Robot sử dụng bánh xe thường sẽ đơn giản hơn các mô hình robot khác không sử dụng bánh xe do robot có bánh xe hầu hết được thiết kế sao cho tất cả các bánh xe luôn tiếp xúc với mặt đất. Với các robot sử dụng 3 bánh xe, các bánh sẽ luôn được tiếp xúc với mặt đất, từ đó đảm bảo việc hoạt động ổn định của bánh xe và robot. Với các robot có từ 4 bánh xe trở lên, hệ thống treo sẽ được sử dụng để đảm bảo tất cả các bánh xe tiếp xúc với mặt đất khi robot gặp địa hình không bằng phẳng.



Hình 1. 6 Mô hình bánh xe cơ bản có thể lựa chọn cho robot nhận dạng và cảnh báo sớm nguy cơ cháy.

(a) Bánh xe cơ bản, hai bậc tự do, quay quanh trục bánh xe (có động cơ) và điểm tiếp xúc.

(b) Bánh xe thầu dầu, hai bậc tự do; xoay quanh một khớp lái đa hướng.

(c) Bánh xe Omni (Swedish 90 độ) hoặc bánh xe Mecanum (Swedish 45 độ).

(d) Bánh xe dạng bóng hoặc bánh xe hình cầu

Đối với robot nhận dạng và cảnh báo sớm nguy cơ cháy, việc sử dụng bánh xe sẽ giúp robot linh hoạt di chuyển trong môi trường hoạt động và đảm bảo tính ổn định của mô hình trong việc nhận dạng và cảnh báo cháy. Việc lựa chọn bánh xe sẽ dựa trên các đặc trưng của bánh:

1.2.1. Thiết kế

Có bốn loại bánh xe chính, như thể hiện trong hình 6. Chúng rất khác nhau về động học và do đó việc lựa chọn loại bánh xe có ảnh hưởng lớn đến động học tổng thể của robot di động. Bánh xe tiêu chuẩn và bánh xe thầu dầu có một trục quay chính và do đó có tính định hướng cao. Để di chuyển theo một hướng khác, trước tiên bánh xe phải được lái dọc theo một trục thẳng đứng. Sự khác biệt chính giữa hai bánh xe này là bánh xe tiêu chuẩn có thể thực hiện chuyển động lái này mà không có tác dụng phụ,

trong khi bánh xe thầu dầu quay quanh một trục lệch, gây ra một lực được truyền tới khung robot trong quá trình điều khiển.

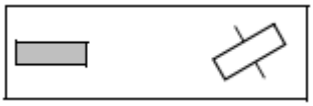
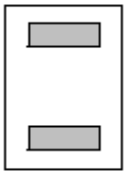
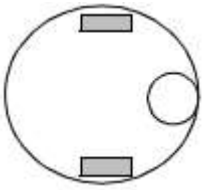
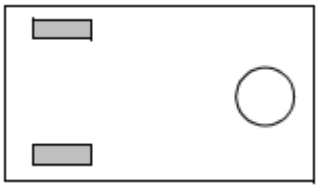
Bánh xe Omni, Mecanum và bánh xe hình cầu đều là những thiết kế ít bị hạn chế về hướng hơn so với bánh xe tiêu chuẩn thông thường. Bánh xe Omni và Mecanum hoạt động như một bánh xe bình thường, nhưng cũng cung cấp lực cản thấp theo hướng khác, đôi khi vuông góc với hướng thông thường như bánh Omni, và đôi khi ở một góc trung gian như bánh xe Mecanum. Các con lăn nhỏ gắn xung quanh chu vi của bánh xe là thụ động và trục chính của bánh xe đóng vai trò là khớp nối chủ động duy nhất. Ưu điểm chính của thiết kế này là, mặc dù vòng quay của bánh xe chỉ được cung cấp năng lượng dọc theo một trục chính (thông qua trục), nhưng bánh xe có thể di chuyển động học với rất ít ma sát dọc theo nhiều quỹ đạo có thể, không chỉ về phía trước và phía sau. Bánh xe hình cầu là một bánh xe đa hướng thực sự, thường được thiết kế sao cho nó có thể được cung cấp năng lượng tích cực để quay theo bất kỳ hướng nào. Một cơ chế để thực hiện thiết kế hình cầu này bắt chước chuột máy tính, cung cấp các con lăn được cung cấp năng lượng tích cực dựa vào bề mặt trên cùng của hình cầu và truyền lực quay. Bất kể loại bánh xe nào được sử dụng, trong rô bốt được thiết kế cho môi trường trên mọi địa hình và rô bốt có nhiều hơn ba bánh xe, hệ thống treo thường được yêu cầu để duy trì sự tiếp xúc của bánh xe với mặt đất [3].

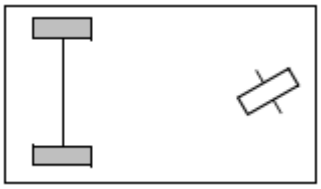
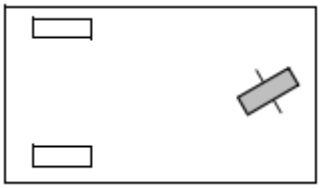
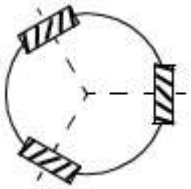
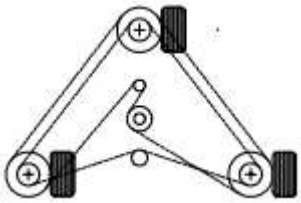
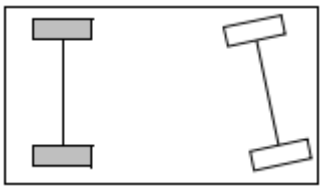
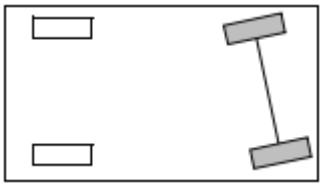
1.2.2. Mô hình

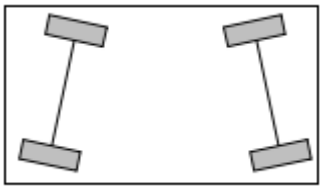
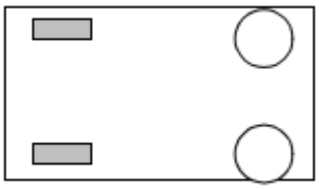
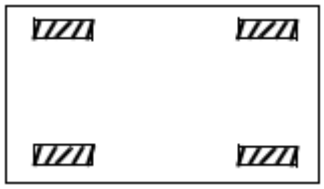
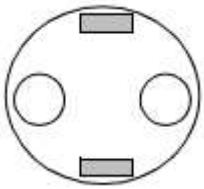
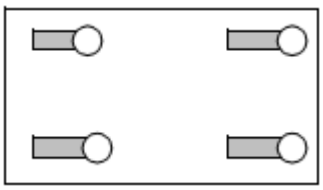
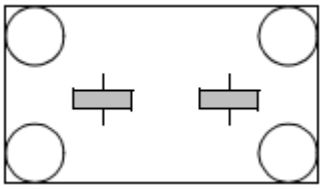
Việc lựa chọn loại bánh xe cho robot di động gắn liền với việc lựa chọn cách sắp xếp bánh xe hoặc hình dạng bánh xe, do đó cần phải xem xét đồng thời hai vấn đề này khi thiết kế cơ chế di chuyển của robot có bánh xe. Ba đặc điểm cơ bản của rô-bốt được điều chỉnh bởi các lựa chọn này: khả năng cơ động, khả năng điều khiển và tính ổn định. Không giống như ô tô, phần lớn được thiết kế cho môi trường tiêu chuẩn hóa cao (mạng lưới đường bộ), robot di động được thiết kế cho các ứng dụng trong nhiều tình huống khác nhau, do robot hoạt động trong nhiều môi trường và yêu cầu khả năng di chuyển khác nhau trong mỗi trường hợp đặc thù nên các loại bánh xe dành cho robot rất đa dạng. Bảng dưới đây đưa ra tổng quan về cấu hình bánh xe được sắp xếp theo số lượng bánh xe. Bảng dưới đây cho thấy cả việc lựa chọn các loại bánh xe cụ thể và cấu hình hình học của chúng trên khung gầm rô-bốt. Bảng này cung cấp sự đa dạng lớn của các cấu hình bánh xe có thể có trong thiết kế robot di động [3].

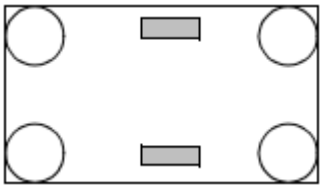
1.2.3. Tính ổn định

Như đã trình bày ở trên, robot dẫn động vi sai hai bánh có thể đạt được độ ổn định tĩnh nếu trọng tâm nằm dưới trục bánh xe. Cye là một robot di động thương mại sử dụng cấu hình bánh xe này. Tuy nhiên, trong những trường hợp thông thường, một giải pháp như vậy đòi hỏi đường kính bánh xe lớn và không thực tế. Thông thường, độ ổn định tĩnh yêu cầu tối thiểu ba bánh xe, với trọng tâm phải nằm trong tam giác được tạo thành bởi bánh xe điểm tiếp xúc với mặt đất của bánh xe. Độ ổn định có thể được cải thiện hơn nữa bằng cách thêm nhiều bánh xe hơn, mặc dù khi số lượng điểm tiếp xúc vượt quá ba, bản chất tĩnh của hình học sẽ yêu cầu một số dạng hệ thống treo linh hoạt trên địa hình không bằng phẳng [3].

Số bánh xe	*Mô hình bánh xe*	*Mô tả*	*Ví dụ thực tế*
2		*Một tay lái ở phía trước, một bánh xe kéo ở phía sau*	*Xe đạp, xe máy*
		*Dẫn động vi sai hai bánh với tâm khối lượng (COM) bên dưới trục*	*Robot Cye*
3		*Dẫn động vi sai trung tâm hai bánh với điểm tiếp xúc thứ ba*	
		*Hai bánh dẫn động độc lập ở phía sau/phía trước, 1 bánh đa hướng không trợ lực ở phía trước/phía sau*	

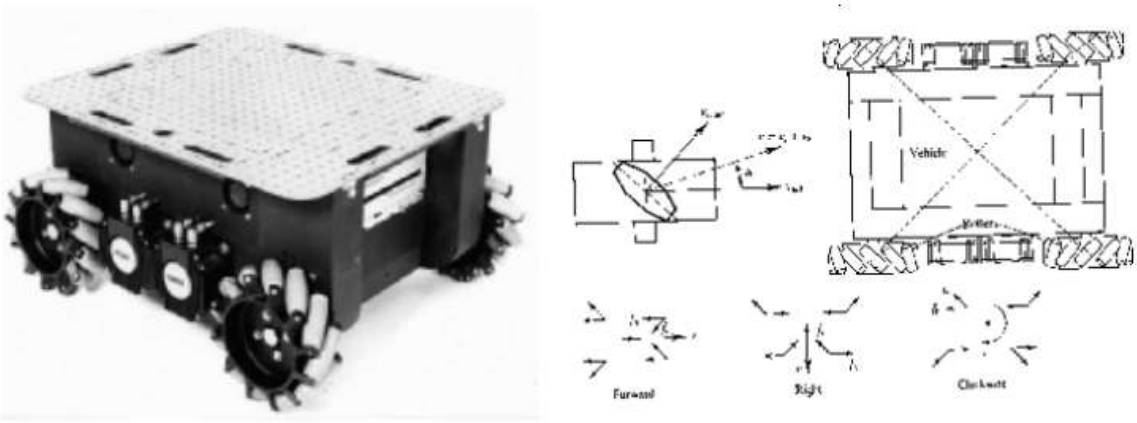
		*Hai bánh xe kéo được kết nối (vi sai) ở phía sau, 1 bánh xe tự do được lái ở phía trước*	
		*Hai bánh tự do ở phía sau, 1 bánh kéo có hướng ở phía trước*	
		*Robot 3 bánh sử dụng bánh xe Omni*	*Stanford wheel Tribolo EPFL, Palm Pilot Robot Kit (CMU)*
		*Robot ba bánh lái và động cơ đồng bộ*	*"Synchro drive" Denning MRV-2, Georgia Institute of Technology, I-Robot B24, Nomad 200*
4		*Hai bánh được dẫn động motor ở phía trước*	*Ô tô dẫn động cầu trước*
		*Hai bánh được dẫn động motor ở phía sau*	*Ô tô dẫn động cầu sau*

		*Bốn bánh lái và dẫn động*	*Four-wheel drive, fourwheel steering Hyperion (CMU)*
		*2 bánh được truyền và dẫn động bằng motor phía trước/sau và 2 bánh đa hướng phía trước/sau*	*Charlie (DMT-EPFL)*
		*Robot 4 bánh Omni/Mecanum*	*Carnegie Mellon Uranus*
		*Robot 4 bánh sử dụng 2 bánh truyền và dẫn động đối xứng, 2 bánh đa hướng đối xứng qua tâm*	*EPFL Khepera, Hyperbot Chip*
		*Robot 4 bánh truyền động kết hợp sử dụng bánh đa hướng*	*Nomad XR4000*
6		*Robot sử dụng 2 bánh truyền và dẫn động ở trục giữa, 4 bánh đa hướng ở mỗi góc*	*First*

		*Robot sử dụng 2 bánh truyền và dẫn động ở hai bên trục giữa, 4 bánh đa hướng mỗi góc*	*Terregator (Carnegie Mellon University)*
Biểu tượng của các bánh xe			
	*Bánh xe đa hướng (không được truyền động từ động cơ)*		
	*Bánh xe Omni/Mecanum được truyền động từ động cơ*		
	*Bánh xe cơ bản (không được truyền động từ động cơ)*		
	*Bánh xe cơ bản (được truyền động từ động cơ)*		
	*Bánh xe đa hướng được truyền động từ động cơ*		
	*Bánh lái cơ bản*		
	*Bánh xe được kết nối*		

1.2.4. Khả năng cơ động

Một số robot đa hướng có thể di chuyển bất kỳ lúc nào theo bất kỳ hướng nào dọc theo mặt đất bất kể hướng của rô-bốt quanh trục thẳng đứng của nó. Mức độ cơ động này yêu cầu các bánh xe có thể di chuyển theo nhiều hướng và do đó rô-bốt đa hướng thường sử dụng bánh xe Omni/Mecanum hoặc bánh xe hình cầu được cung cấp năng lượng. Một ví dụ điển hình robot Uranus, robot này sử dụng bốn bánh xe Mecanum để xoay và dịch một cách độc lập và không bị ràng buộc [3].



Hình 1. 7 Robot Carnegie Mellon Uranus, sử dụng 4 bánh xe Mecanum

1.2.5. Khả năng điều khiển

Nhìn chung có một sự đối lập giữa khả năng kiểm soát và khả năng cơ động. Ví dụ: các thiết kế đa hướng như cấu hình bánh xe bốn bánh yêu cầu quá trình xử lý quan trọng để chuyển đổi vận tốc quay và tịnh tiến mong muốn thành các lệnh bánh xe riêng lẻ. Hơn nữa, các thiết kế đa hướng như vậy thường có mức độ tự do cao hơn trong bài toán điều khiển. Việc điều khiển robot đa hướng theo một hướng di chuyển cụ thể cũng khó khăn hơn và thường kém chính xác hơn khi so sánh với các thiết kế ít cơ động hơn. Với hệ dẫn động bốn bánh, chẳng hạn như robot Uranus có bốn bánh của Mecanum, vấn đề còn khó hơn vì cả bốn bánh phải được dẫn động với tốc độ chính xác như nhau để robot di chuyển trên một đường thẳng chính xác.

Như vậy có thể thấy, không có cấu hình truyền động "lý tưởng" nào đồng thời tối đa hóa độ ổn định, khả năng cơ động và khả năng kiểm soát. Mỗi ứng dụng rô-bốt di động đặt ra các ràng buộc duy nhất đối với vấn đề thiết kế rô-bốt và nhiệm vụ đặt ra chính là chọn bánh xe và kết cấu khung robot phù hợp [3].

1.3. Mục tiêu và đối tượng nghiên cứu

Điều khiển robot di chuyển linh hoạt trong phạm vi hoạt động, sử dụng camera để nhận dạng đám cháy.

Với mục tiêu điều khiển được robot sử dụng 3 bánh Omni có độ chính xác và tin cậy, tôi chọn mô hình robot có 3 bánh xe tạo với nhau một góc 120 độ. Khi 3 bánh tạo với nhau góc 120 độ, mục tiêu đặt ra là phải điều khiển được robot di chuyển liên tục trong không gian hoạt động, điều khiển được tốc độ di chuyển và nắm được quãng đường chuyển động của robot. Để robot có thể di chuyển với độ chính xác tin cậy, việc hiệu

chỉnh công suất cho từng động cơ và nắm được số vòng xoay được mỗ bánh rất quan trọng do chúng ảnh hưởng trực tiếp đến tọa độ của robot trong phạm vi hoạt động.

1.4. Nhiệm vụ của đề tài và phạm vi nghiên cứu

1.4.1. Nhiệm vụ của đề tài

Để thực hiện mục tiêu trên, đề tài "Robot di động nhận dạng và cảnh báo sớm đám cháy sử dụng trí tuệ nhân tạo" cần tập trung vào giải quyết những vấn đề sau:

- *Xây dựng mô hình 3D cho robot*
- *Lựa chọn vi điều khiển và mạch xử lý phù hợp với nhiệm vụ của robot*
- *Phân tích động học của bánh xe Omni và robot 3 bánh sử dụng bánh xe Omni*
- *Điều khiển robot 3 bánh sử dụng bánh xe Omni chạy theo quỹ đạo chuyển động*
- *Xây dựng mô hình nhận dạng và cảnh báo sớm nguy cơ cháy sử dụng trí tuệ nhân tạo*
- *Thực nghiệm và đưa ra kết quả*

1.4.2. Phạm vi nghiên cứu

- *Robot được thiết kế nhỏ gọn phù hợp với môi trường di chuyển'*
- *Điều khiển robot theo quỹ đạo chuyển động và nắm được quãng đường di chuyển*
- *Xây dựng mô hình nhận dạng nguy cơ cháy cho robot*

1.4.3. Phương pháp nghiên cứu

- *Sử dụng các phương pháp tính toán động học để tính toán cho robot*
- *Ứng dụng việc đọc tín hiệu encoder để tính toán quãng đường và tốc độ của robot bằng vi điều khiển*
- *Sử dụng kết quả thực nghiệm để kiểm chứng khả năng di chuyển của robot theo quỹ đạo*

1.5. Kết luận chương

Hiện nay, trên thế giới đã có rất nhiều công trình nghiên cứu và thiết kế các loại robot có thể giám sát và cảnh báo với nhiều ứng dụng quan trọng trong các ngành công nghiệp và nông nghiệp hiện đại. Trong thực tế, việc nghiên cứu thiết kế và chế tạo robot giám sát và cảnh báo cháy có sử dụng bánh xe sẽ mang tới sự đa năng và hiệu quả cao nhất các ứng dụng trong thực tế các nghành công nghiệp và nông nghiệp hiện đại, tuy nhiên các mẫu robot này chưa mang lại độ linh hoạt và khả năng cảnh báo sớm. Việc nghiên cứu và phát triển robot giám sát 3 bánh có sử dụng bánh Omni là hoàn toàn thiết thực và hiệu quả cao. Do đó, đề tài Robot di động nhận dạng và cảnh báo sớm đám

cháy sử dụng trí tuệ nhân tạo của tôi sẽ là robot sử dụng bánh xe Omni để linh hoạt trong khả năng di chuyển và ứng dụng trí tuệ nhân tạo trong nhận dạng.

Trong chế tạo và điều khiển sẽ đề cập đến việc thiết kế robot sao cho phù hợp với mô hình động học và tính toán quỹ đạo của robot khi sử dụng bánh xe Omni. Việc điều khiển công suất và đọc tín hiệu từ encoder sẽ nằm trong phần điều khiển, trong đó đầu vào sẽ là tín hiệu encoder, từ đó nắm bắt được tốc độ góc, số vòng quay tại mỗi động cơ và sẽ cung cấp tín hiệu điều khiển động cơ dựa trên tính toán để robot di chuyển theo đúng bài toán động học.

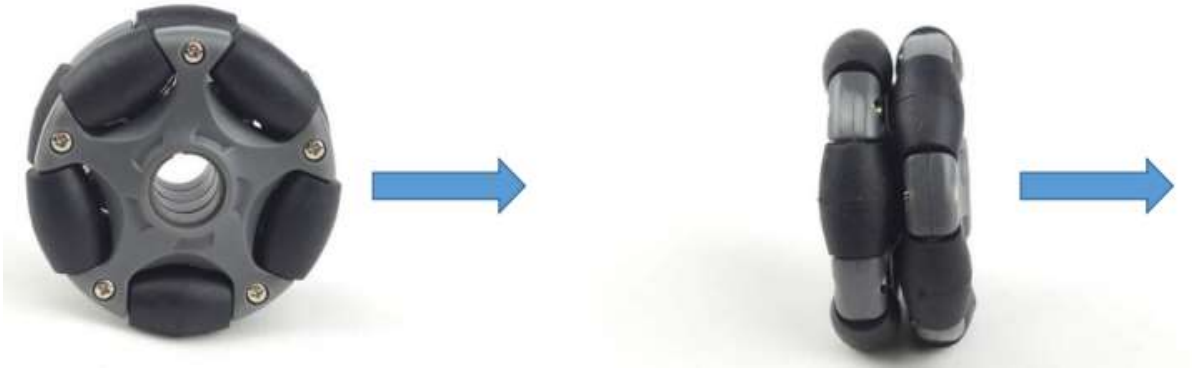
Trong nhận dạng và cảnh báo sẽ đề cập đế việc xây dựng mô hình nhận dạng nguy cơ cháy cho robot sử dụng trí tuệ nhân tạo. Trong đó, đầu vào sẽ là ảnh thu thập được từ camera, qua quá trình tính toán và xử lý của máy tính nhúng, kết quả đầu ra sẽ là tỷ lệ phần trăm của nguy cơ cháy, nếu tỷ lệ này vượt ngưỡng cho phép, robot sẽ đưa ra cảnh báo.

Cụ thể trong đề tài, robot được chia làm hai hệ thống, một hệ thống sẽ đảm bảo việc di chuyển của robot, hệ thống này sử dụng vi điều khiển, mạch cầu và các motor để điều khiển hướng di chuyển của robot; một hệ thống sử dụng máy tính nhúng sẽ thực hiện giám sát và cảnh báo nguy cơ. Hai hệ thống này có mối liên hệ với nhau khi xảy ra nguy cháy, khi đó các thông số từ máy tính nhúng tính toán được sẽ được gửi về encoder để robot di chuyển đến vị trí của nguy cơ cháy theo quỹ đạo mong muốn.

CHƯƠNG 2: PHÂN TÍCH ĐỘNG HỌC VÀ ĐỘNG LỰC HỌC ROBOT 3 BÁNH OMNI

2.1. Phân tích bánh xe Omni

Mô hình bánh xe Omni:



Hình 2. 1 Hai hướng di chuyển của bánh xe Omni 90 độ

Robot sử dụng bánh xe bánh Omni 90 độ thường được thiết kế dưới dạng 3 bánh hoặc 4 bánh, mỗi thiết kế đều có ưu nhược điểm khác nhau:

a. Robot 3 bánh sử dụng bánh xe Omni 90 độ:

+ Ưu điểm:

- *Ma sát tốt hơn và cân bằng trên mặt phẳng hơn so với robot 4 bánh sử dụng bánh xe Omni.*
- *Các bánh luôn chạm đất mà không cần sử dụng hệ thống treo, từ đó tránh sai sót trong việc tính toán động học và động lực học robot.*

+ Nhược điểm:

- *Do 3 trục bánh xe lệch với nhau một góc 120 độ, do đó robot sẽ không sử dụng tốt hiệu suất động cơ*
- *Robot 3 bánh sử dụng bánh xe Omni khó khăn trong việc tính toán động học và động lực học hơn robot 4 bánh sử dụng bánh xe Omni.*
-

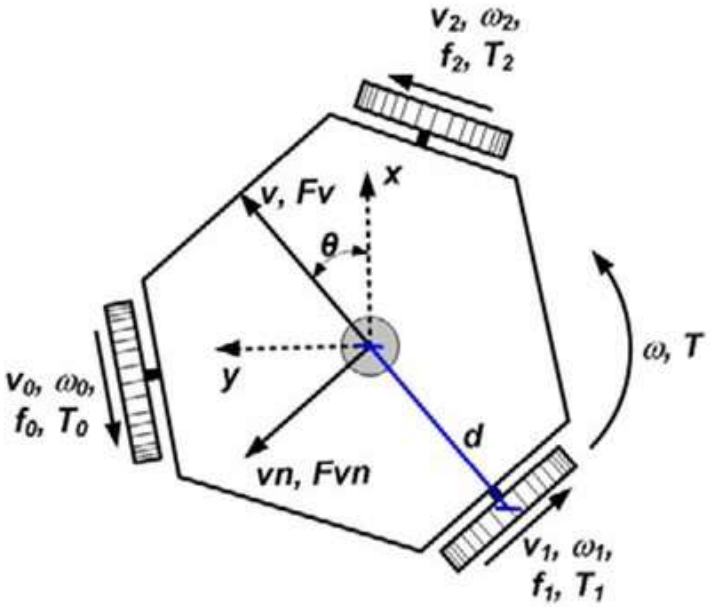
b. Robot 4 bánh sử dụng bánh xe Omni 90 độ:

+Ưu điểm:

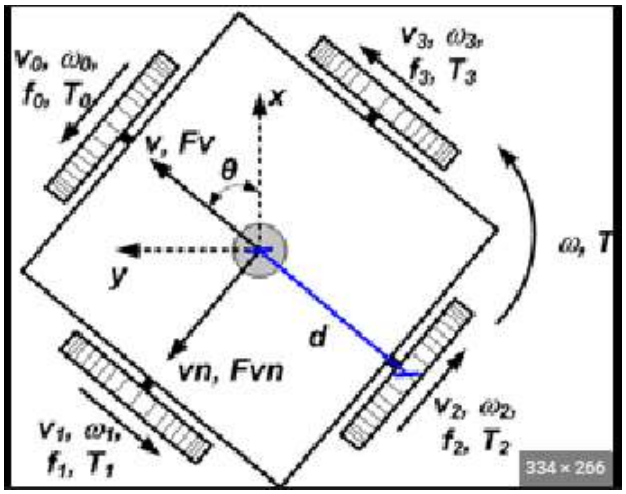
- *Tận dụng tốt hiệu suất động cơ do có trục các bánh xe tạo với nhau góc 90 độ*
- *Bài toán động học và động lực học đơn giản hơn robot 3 bánh sử dụng bánh xe Omni*

+Nhược điểm:

- *Khó để đảm bảo tiếp xúc cho cả 4 bánh khi di chuyển, nhất là khi mặt đường di chuyển không bằng phẳng, từ đó gây ra sai sót trong tính toán động học và động lực học*
- *Khó khăn trong việc thiết kế hệ thống treo đảm bảo cho 4 bánh xe luôn chạm đất*



Hình 2. 2 Robot 3 bánh sử dụng bánh xe Omni [4]

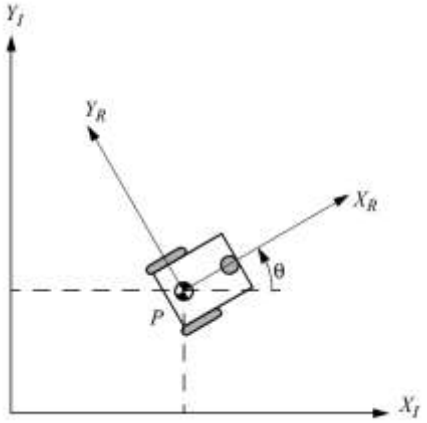


Hình 2. 3 Robot 4 bánh sử dụng bánh xe Omni [5]

2.2. Xác định vị trí Robot

Trong quá trình phân tích cần mô hình hóa rô-bốt như một vật rắn trên các bánh xe, hoạt động trên mặt phẳng nằm ngang. Tổng kích thước của khung rô-bốt này trên

mặt phẳng là ba, hai dành cho vị trí trong mặt phẳng và một dành cho hướng dọc theo trục thẳng đứng, trực giao với mặt phẳng. Có thêm mức độ tự do và linh hoạt do trục bánh xe, khớp lái bánh xe và khớp bánh xe. Tuy nhiên, đối với khung gầm rô-bốt, chúng ta chỉ đề cập đến phần thân của rô-bốt, bỏ qua các khớp nối và bậc tự do bên trong rô-bốt và các bánh xe của rô-bốt [3].

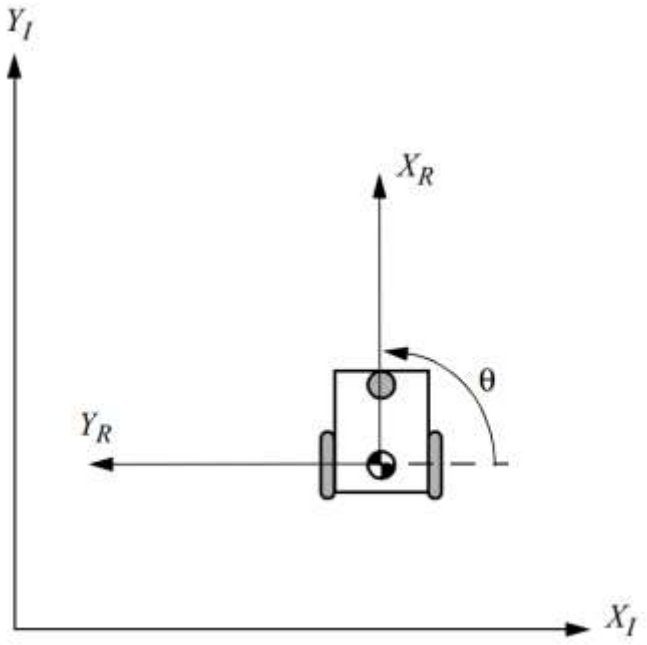


Hình 2. 4 Hệ trục tọa độ tổng thể và hệ trục tọa độ cơ sở của Robot

Để xác định vị trí của robot trên mặt phẳng ta thiết lập mối quan hệ giữa hệ trục tọa độ tổng thể của mặt phẳng và hệ trục tọa độ cơ sở của robot như hình 11. Các trục X_I và Y_I xác định một cơ sở quán tính tùy ý trên mặt phẳng làm hệ quy chiếu tổng thể từ một gốc tọa độ $O:\{X_I,Y_I\}$ nào đó. Để xác định vị trí của rô-bốt, chọn một điểm P trên khung rô-bốt làm điểm tham chiếu vị trí của nó. Hệ trục tọa độ cơ sở của robot xác định hai trục tương ứng với P trên khung robot. Vị trí của P trong hệ trục tọa độ tổng thể được xác định bởi tọa độ x và y, và sự khác biệt về góc giữa hệ trục tọa độ tổng thể và hệ trục tọa độ cơ sở được cho bởi θ. Chúng ta có thể mô tả tư thế của rô-bốt dưới dạng một vectơ với ba yếu tố này. Việc sử dụng chỉ số I để làm rõ cơ sở của tư thế này là hệ trục tọa độ tổng thể:

$$\zeta_I = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} \qquad (2.1)$$

Để mô tả chuyển động của rô-bốt dưới dạng các chuyển động thành phần, cần xác định chuyển động dọc theo các trục của hệ trục tọa độ tổng thể thành chuyển động dọc theo các trục của hệ trục tọa độ cơ sở của rô-bốt. Việc xác định này được thực hiện bằng cách sử dụng ma trận xoay trực giao:



Hình 2. 5 Robot di động được căn chỉnh theo hệ trục tọa độ tổng thể.

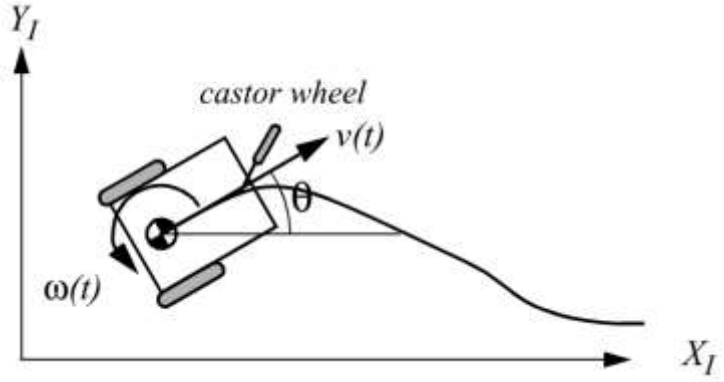
$$R(\theta)=\begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 \end{bmatrix} \tag{2.2}$$

Ma trận này có thể được sử dụng để xác định chuyển động trong hệ tọa độ chiếu tổng thể $\{X_I, Y_I\}$ *sang chuyển động theo hệ tọa độ cơ sở* $\{X_R, Y_R\}$. *Ở đây, R(theta) phụ thuộc vào giá trị của* θ*:*

$$\dot{\xi}_R = R\left(\frac{\pi}{2}\right)\dot{\xi}_I \tag{2.3}$$

Ví dụ, xét robot trong hình 12. Đối với robot này, bởi vì $\theta=\frac{\pi}{2}$*, nên có thể dễ dàng tính toán ma trận quay tức thời R:*

$$R\left(\frac{\pi}{2}\right)=\begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \tag{2.4}$$



Hình 2. 6 Một robot dẫn động vi sai trong hệ tọa độ tổng thể.

Cho trước một vận tốc $(\dot{x}, \dot{y}, \dot{\theta})$ *trong hệ quy chiếu tổng thể, chúng ta có thể tính toán các thành phần của chuyển động dọc theo các trục cục bộ* X_R *và* Y_R *của robot. Trong trường hợp này, do góc cụ thể của robot, chuyển động dọc theo* X_R *bằng* $\dot{y}$ *và chuyển động dọc theo* Y_R *bằng* $-\dot{x}$ *:*

$$\dot{\zeta}_R = R\left(\frac{\pi}{2}\right)\dot{\zeta}_I = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \dot{y} \\ -\dot{x} \\ \dot{\theta} \end{bmatrix} \qquad (2.5)$$

2.3. Phân tích động học robot 3 bánh Omni

Mô hình động học của Robot 3 bánh sử dụng bánh xe Omni biểu diễn theo tọa hệ độ (x, y, θ)*, trong đó các thông số của Robot được mô tả như sau:*

(x, y, θ)*- Vị trí của robot (x, y) và* θ *là góc được định nghĩa tại mặt trước của robot*

d[m] - khoảng cách giữa bánh xe và tâm của robot

$v_0, v_1, v_2[m/s]$ *- vận tốc tuyến tính của bánh xe*

$\omega_0, \omega_1, \omega_2[rad/s]$ *- vận tốc góc của bánh xe*

f_0, f_1, $f_2[N]$ *- lực kéo của bánh xe*

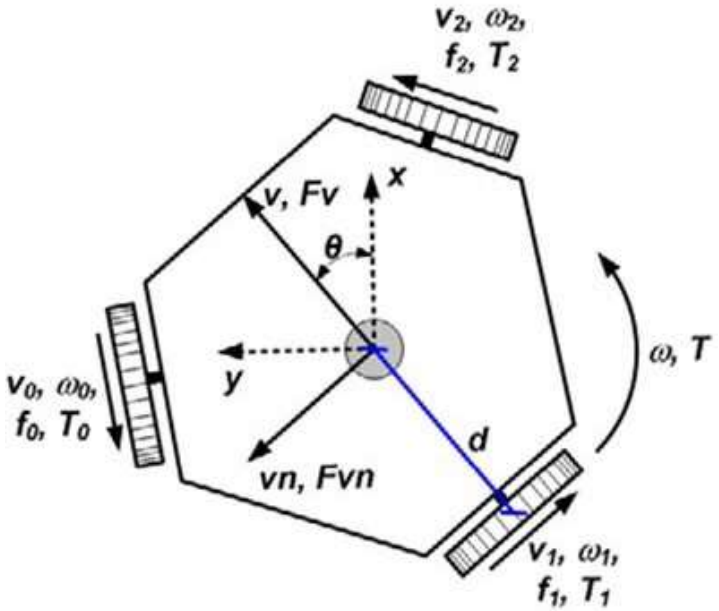
$T_0, T_1, T_2[N.m]$ *- Mô men quay tại các bánh xe*

$v, vn[m/s]$ *- vận tốc tuyến tính của robot*

$\omega[rad/s]$ - vận tốc góc của robot

$F_v, F_{vn}[N]$ - lực kéo của robot theo v và vn

$T[N.m]$ - Mô men quay của robot (phụ thuộc vào ω)



Hình 2. 7 Bài toán động học cho robot 3 bánh Omni

Với hàm vận tốc và gia tốc được mô tả bởi:

$$\begin{aligned} v_x(t) &= dx(t)/dt \\ v_y(t) &= dy(t)/dt \\ \omega(t) &= d\theta(t)/dt \end{aligned} \qquad (2.6)$$

Từ đó ta có công thức chuyển đổi giữa vận tốc tuyến tính v_x và v_y trong hệ tọa độ gốc sang vận tốc v và vn trên tọa độ của robot:

$$X_R = \begin{bmatrix} v(t) \\ vn(t) \\ \omega(t) \end{bmatrix};\ X_0 = \begin{bmatrix} v_x(t) \\ v_y(t) \\ \omega(t) \end{bmatrix};\ X_R = \begin{bmatrix} cos(\theta(t)) & sin(\theta(t)) & 0 \\ -sin(\theta(t)) & cos(\theta(t)) & 0 \\ 0 & 0 & 1 \end{bmatrix}.X_0 \qquad (2.7)$$

Mối quan hệ giữa vận tốc của bánh xe và vận tốc của robot được mô tả như sau:

$$\begin{bmatrix} v_0(t) \\ v_1(t) \\ v_2(t) \end{bmatrix} = \begin{bmatrix} -\sin(\pi/3) & \cos(\pi/3) & d \\ 0 & -1 & d \\ \sin(\pi/3) & \cos(\pi/3) & d \end{bmatrix} . \begin{bmatrix} v(t) \\ vn(t) \\ \omega(t) \end{bmatrix} \quad (2.8)$$

Từ công thức vận tốc của từng bánh xe, áp dụng ma trận động học nghịch của robot từ phương trình (2.8) ta thu được:

$$v(t) = (\sqrt{3}/3).(v_2(t) - v_0(t)) \quad (2.9)$$

$$vn(t) = (1/3).(v_2(t) + v_0(t)) - (2/3).v_1(t) \quad (2.10)$$

$$\omega(t) = (1/(3.d)).(v_0(t) + v_1(t) + v_2(t)) \quad (2.11)$$

2.4. Phân tích động lực học robot 3 bánh Omni

Các phương trình động lực học của robot được mô tả bởi các công thức sau [4]:

$$M.\frac{dv(t)}{dt} = \sum F_v(t) - F_{Bv}(t) - F_{Cv}(t) \quad (2.12)$$

$$M.\frac{dvn(t)}{dt} = \sum F_{vn}(t) - F_{Bvn}(t) - F_{Cvn}(t) \quad (2.13)$$

$$J.\frac{d\omega(t)}{dt} = \sum T(t) - T_{B\omega}(t) - T_{\omega}(t) \quad (2.14)$$

trong đó, các thông số của phương trình động lực học được mô tả như sau:

$M[kg]$ *- khối lượng*

$J[kg.m^2]$ *- mô men quán tính*

$F_{Bv}, F_{Bvn}[N]$ *- lực ma sát nhớt*

$T_{B\omega}[N.m]$ *- mô men xoắn nhớt khi robot xoay trục*

$F_{Cv}, F_{Cvn}[N]$ *- lực ma sát Cu lông của robot*

$T_{C\omega}[N.m]$ *- mô men xoắn bởi lực ma sát Cu lông khi robot xoay trục*

Lực ma sát được mô tả như sau:

$F_{Cv}(t) = C_v.sign(v(t))$, $F_{Cvn}(t) = C_{vn}.sign(vn(t))$ *và* $T_{C\omega}(t) = C_{\omega}.sign\omega(t)$ *trong đó* C_v , $C_{vn}[N]$ *là hệ số ma sát theo* v , vn *và* $C_{\omega}[N.m]$ *là hệ số ma sát theo* ω

Tương quan giữa mô-men qay với lực tại tác dụng lên các bánh xe được mô tả bởi những phương trình sau:

$$\sum F_v(t) = (f_2(t) - f_0).sin(\pi/3) \quad (2.15)$$

$$\sum F_{vn}(t) = -f_1(t) + (f_2(t) + f_0(t)).cos(\pi/3) \quad (2.16)$$

$$\sum T(t) = (f_0(t) + f_1(t) + f_2(t)).d \quad (2.17)$$

Lực kéo trên mỗi bánh xe liên hệ với mô-men kéo được xác định với dòng điện của động cơ được mô tả theo những phương trình sau:

$$f_j(t) = T_j(t)/r \quad (2.18)$$

$$T_j(t) = l.K_t.i_j(t) \quad (2.19)$$

Trong đó:

l *- tỷ số truyền của hộp số giảm tốc*

$r[m]$ *- bán kính bánh xe*

$K_t[N.m/A]$ *- mô men xoắn của động cơ*

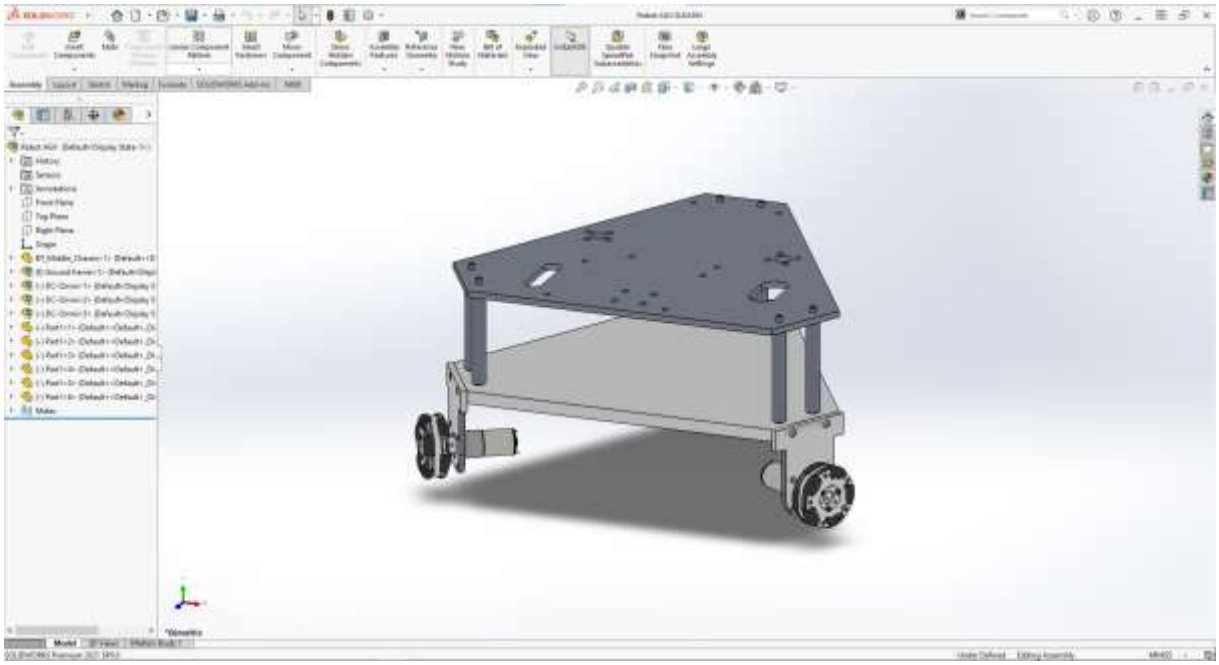
$i_j[A]$ *- dòng điện qua động cơ*

CHƯƠNG 3: MÔ HÌNH 3D, THUẬT TOÁN ĐIỀU KHIỂN VÀ NHẬN DẠNG CỦA ROBOT

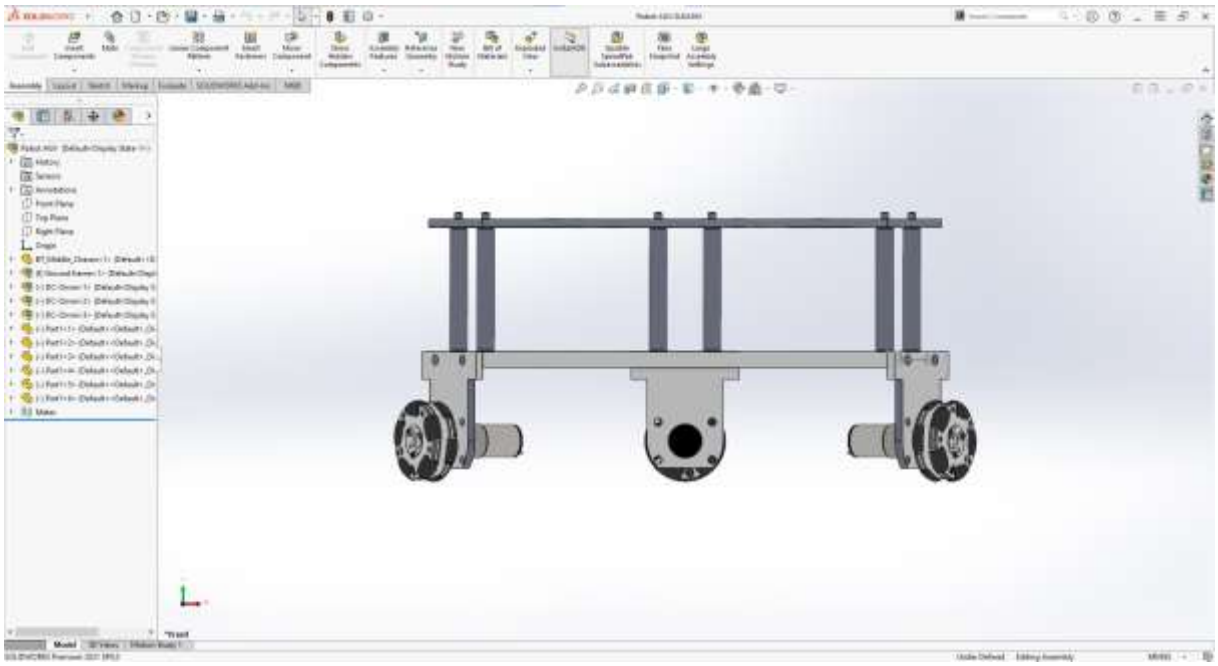
3.1. Xây dựng mô hình 3D cho robot

Trong thiết kế 3D robot có sử dụng bánh xe Omni - là một loại bánh xe đa hướng, có khả năng xoay 360 độ. Robot được thiết kế với cấu trúc vững chắc, phù hợp với môi trường làm việc khác nhau, loại bánh xe này được ứng dụng trong hệ thống băng chuyền hoặc dây chuyền sản xuất sản phẩm điện tử. Chẳng hạn như vận chuyển đường sắt, bàn xoay máy tính, bánh xe robot, …

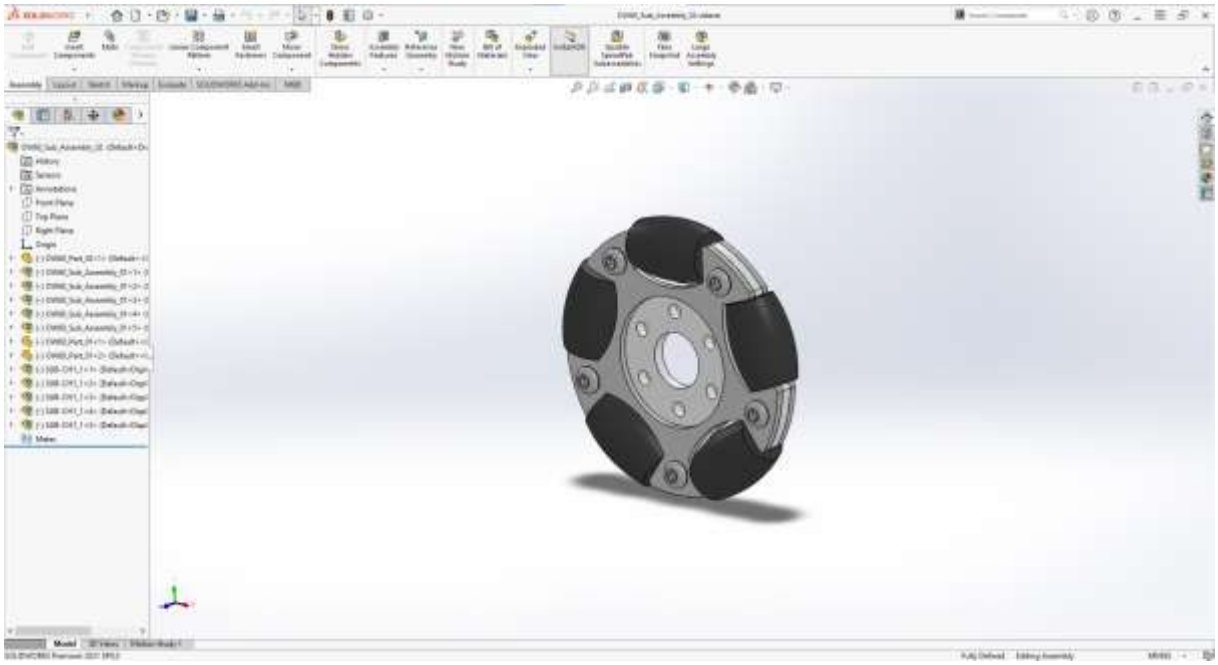
Ưu điểm của bánh Omni là được sử dụng cho các thiết kế Robot chuyên nghiệp, cần độ linh hoạt, chuyển hướng nhanh. Thông thường, các bánh xe Omni có cấu tạo bằng nhựa ABS và cao su cao cấp với chất lượng và độ bền cao. Khung bánh xe được làm bằng nhôm giúp bảo vệ bánh xe và robot trước những tác động lớn. Đồng thời đảm bảo an toàn cho các động cơ bên trong.



Hình 3. 1 Mô hình 3D robot 3 bánh Omni



Hình 3. 2 Mô hình tổng thể của Robot khi nhìn ngang

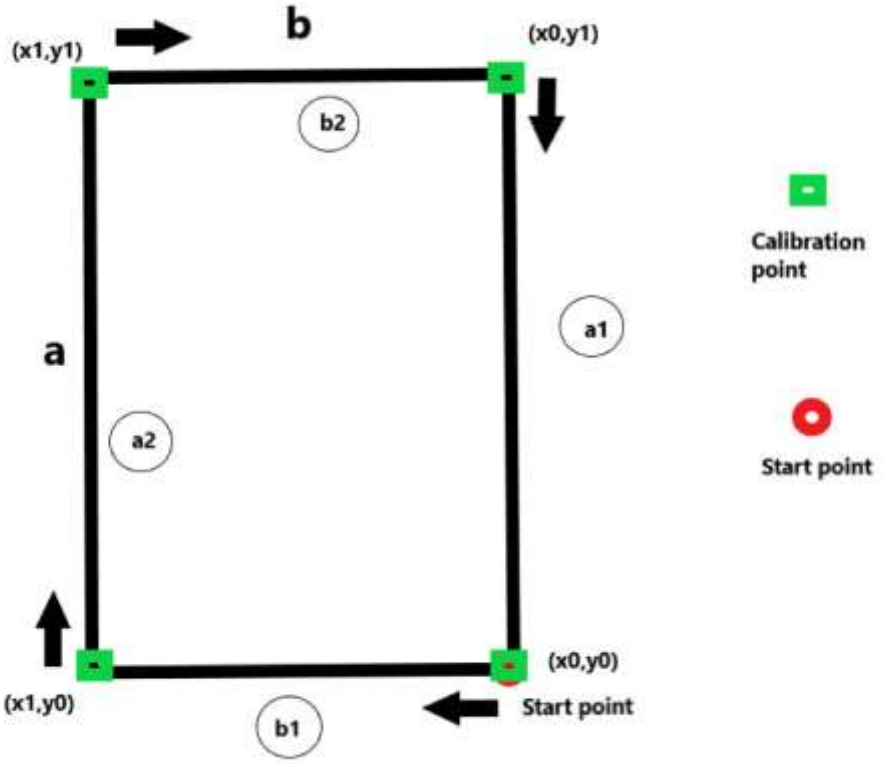


Hình 3. 3 Mô hình bánh xe Omni

Với các bánh xe Omni, dòng robot này có thể tịnh tiến và quay tại chỗ theo hai chiều. Điều khiển nó đơn giản nhờ phần mềm nhưng vẫn giữ tính linh hoạt. Những khả năng vượt trội ở trên giúp phát huy được thế mạnh của robot.

3.2. Thuật toán điều khiển robot

3.2.1. Điều khiển Robot tự động di chuyển trong theo tọa độ khi chưa nhận dạng được nguy cơ cháy



Hình 3. 4 Quỹ đạo di chuyển của robot khi không phát hiện nguy cơ cháy

Ta có các thông số của robot như sau:

R – bán kính bánh

n – số encoder mỗi 1 vòng xoay

a-b - độ dài quỹ đạo theo chiều ngang và dọc của không gian di chuyển của robot

từ đó, việc điều khiển sẽ phụ thuộc vào số xung đếm được trên encoder của từng động cơ. Khi không nhận dạng được nguy cơ cháy, robot sẽ di chuyển xung quanh khu vực cần giám sát để có thể thu thập hình ảnh xung quanh, đảm bảo việc giám sát trong toàn bộ khu vực hình chữ nhật có chiều dài "a" và chiều rộng "b". Khi đó các bước tính toán và điều khiển robot sẽ như sau:

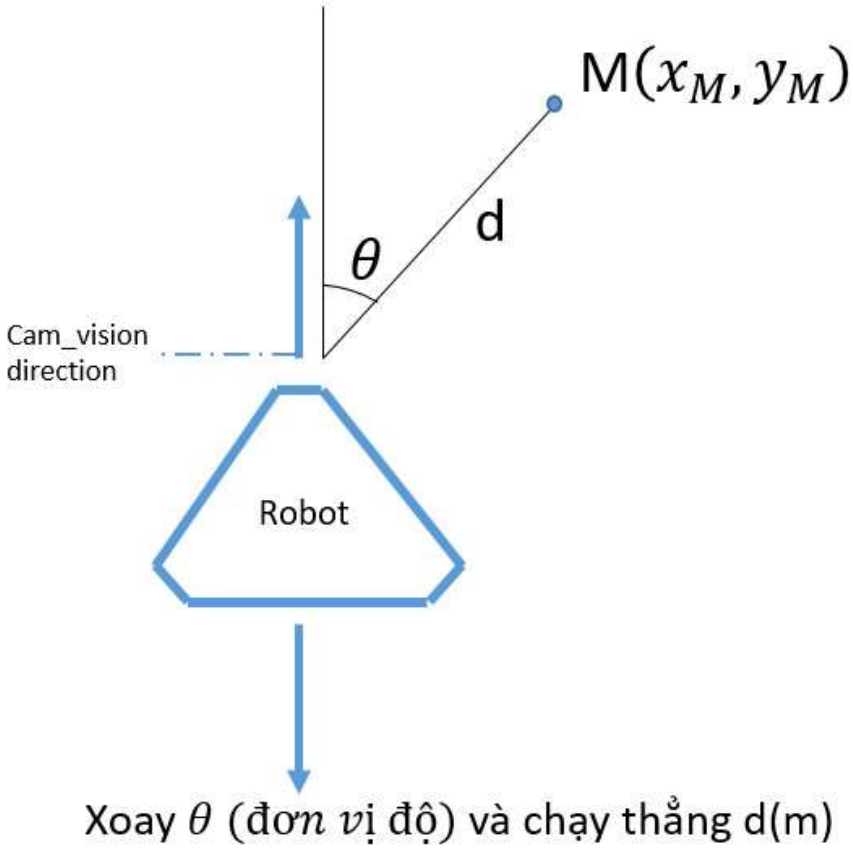
+ Số xung khi quỹ đạo thẳng, dài a(m)

$$npa = \frac{a}{\pi 2R} \times n \times \frac{2\sqrt{3}}{3} \qquad (3.1)$$

+ Số xung khi quỹ đạo thẳng, dài b(m)

$$npb = \frac{b}{\pi 2R} \times n \times \frac{2\sqrt{3}}{3} \quad (3.2)$$

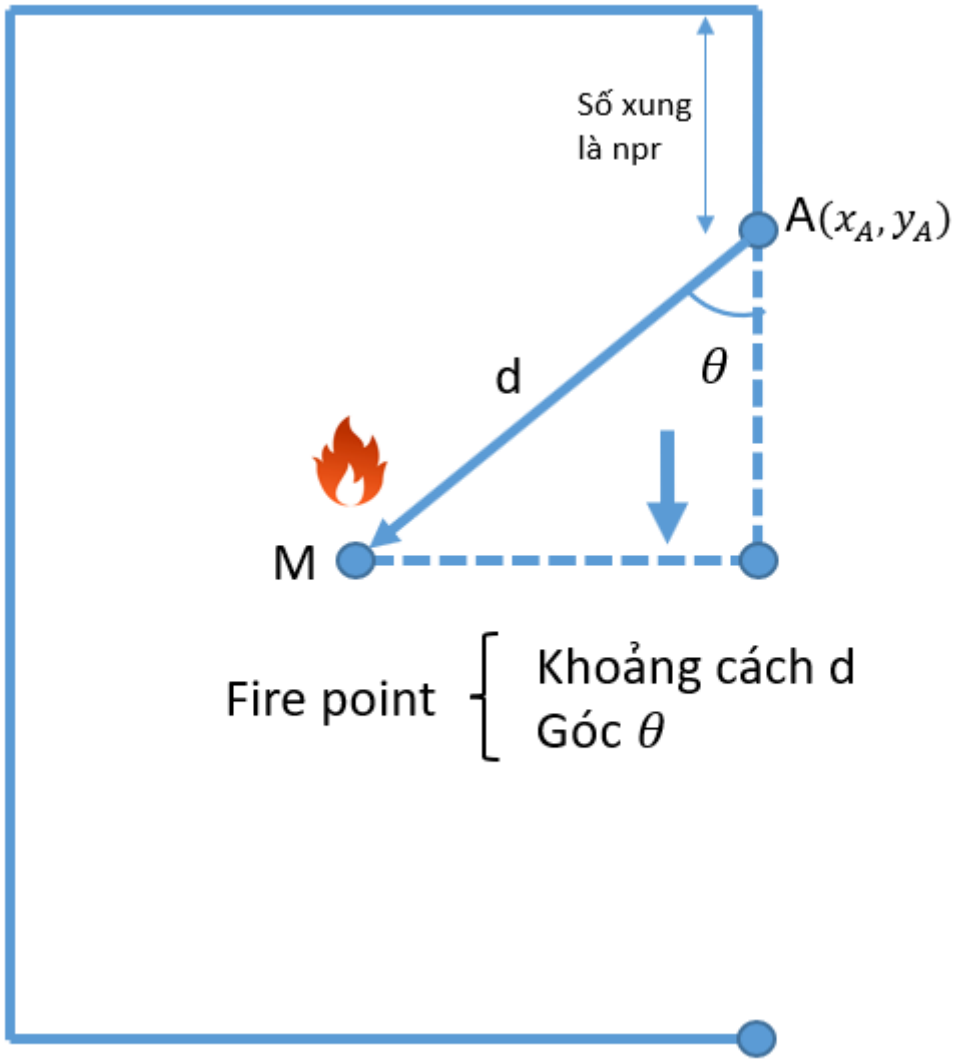
3.2.2. Điều khiển robot tự động di chuyển lại gần nguy cơ cháy khi phát hiện có nguy cơ cháy



Hình 3. 5 Robot khi phát hiện và tính toán vị trí nguy cơ cháy

Khi robot phát hiện ra nguy cơ cháy, hàm ngắt của vi điều khiển sẽ được kích hoạt và di chuyển dựa theo thông số vị trí được cung cấp. Với giá trị d và θ được trả về từ thuật toán đo khoảng cách, ta có $M(x_M, y_M)$ là điểm nhận dạng lửa với:

$$\begin{cases} x_M = x_A - d.cos(\theta) \\ y_M = y_A - d.cos(\theta) \end{cases} \quad (3.3)$$



Hình 3. 6 Quỹ đạo di chuyển của robot khi nhận dạng được nguy cơ cháy

Với A là điểm mà tại đó robot phát hiện có nguy cơ cháy

TH1: A trên quỹ đạo (a1)

$$A\left(x_0, a - \frac{npr}{n}.2\pi R\right) \quad (3.4)$$

TH2: A trên quỹ đạo (a2)

$$A\left(x_1, \frac{npr}{n}.2\pi R\right) \quad (3.5)$$

TH3: A trên quỹ đạo (b1)

$$A\left(-\frac{npr}{n}.2\pi R, y_0\right) \quad (3.6)$$

TH4: A trên quỹ đạo (b2)

$$A\left(b-\frac{npr}{n}.2\pi R, y_1\right) \quad (3.7)$$

Yêu cầu của Robot được đưa ra là di chuyển đến vị trí có nguy cơ cháy để nâng cao độ chính xác và đưa ra cảnh báo. Khi đó, thuật toán điều khiển robot sẽ như sau:

** Xoay góc* θ

D là đường kính Robot (tính cả khi đã lắp bánh)

➔ *Xoay* θ *khi* $\begin{cases} v_1 = v_2 = v_3 \\ s_1 = s_2 = s_3 = \dfrac{\theta}{360}\pi D \end{cases}$

Với np *là number of pulses*

R *bán kính bánh*

** chạy thẳng d*

$$\begin{cases} v_2 = v_3 \\ v_1 = 0 \\ np = \dfrac{d}{2\pi R} n \times \dfrac{2\sqrt{3}}{3} \end{cases} \quad (3.8)$$

Điều khiển robot quay lại x_A, y_A *khi muốn quay trở lại về quỹ đạo ban đầu*

** chạy thẳng (-d) với bánh xe xoay ngược chiều với lúc đi vector AM*

$$\begin{cases} v_2 = v_3 \\ v_1 = 0 \\ np = \dfrac{d}{2\pi R} n \times \dfrac{2\sqrt{3}}{3} \end{cases} \quad (3.9)$$

** Xoay góc (-θ) với bánh xe xoay ngược chiều khi xoay góc θ*

$$\begin{cases} v_1 = v_2 = v_3 \\ np = \dfrac{\theta}{360} \pi D \dfrac{1}{2\pi R} n \times \dfrac{2\sqrt{3}}{3} \end{cases} \quad (3.10)$$

3.3. Mô hình nhận dạng nguy cơ cháy

3.3.1. Mô hình Yolov5 trong nhận dạng đám cháy

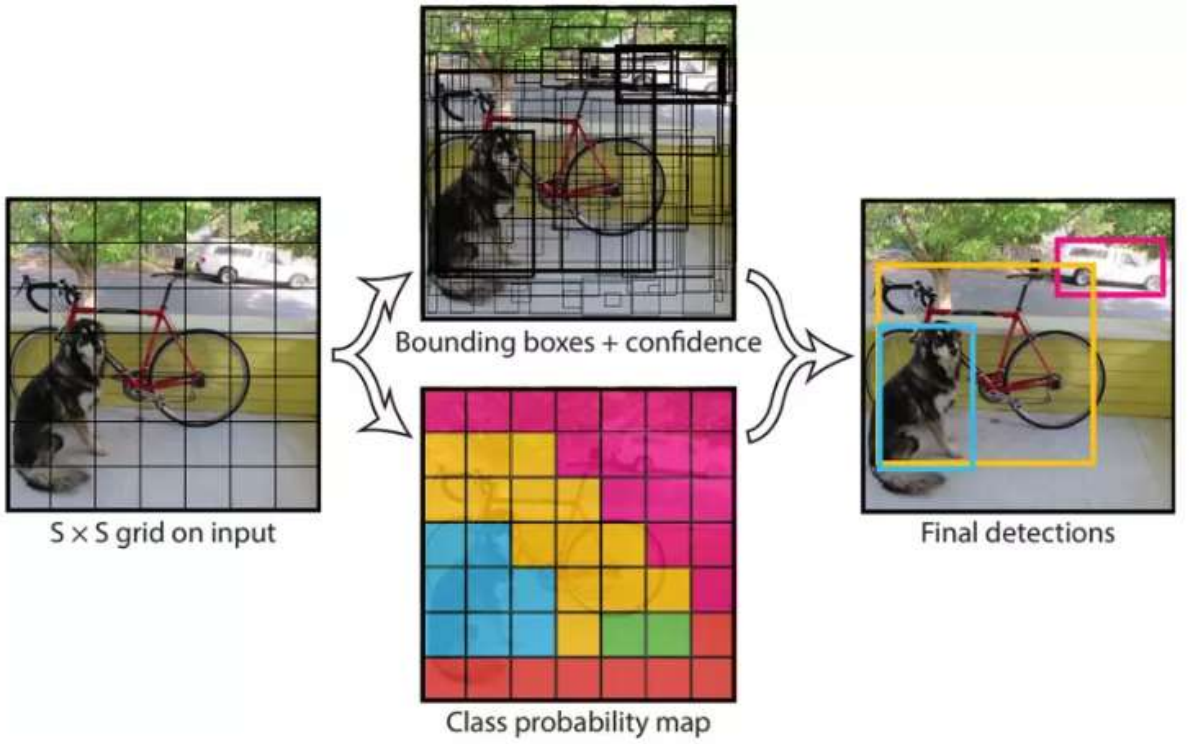
Yolo là một mô hình mạng CNN cho việc phát hiện, nhận dạng, phân loại đối tượng. Yolo được tạo ra từ việc kết hợp giữa các convolutional layers và connected layers. Trong đó các convolutional layers sẽ trích xuất ra các feature của ảnh, còn full-connected layers sẽ dự đoán ra xác suất đó và tọa độ của đối tượng. YOLO có thể không phải là thuật toán tốt nhất nhưng chính là thuật toán có tốc độ nhanh nhất trong các lớp mô hình object detection. Nó có thể đạt đến gần tốc độ real-time mà độ chính xác hầu như là giảm không đáng kể. YOLOv5 là một mô hình Object Detection thuộc họ mô hình YOLO. 3 phiên bản YOLO đầu tiên được phát triển bởi Joseph Redmon. Sau đó, Alexey Bochkovskiy cho ra mắt YOLOv4 với sự cải thiện cả về tốc độ cũng như độ chính xác. Và rồi YOLOv5 được công bố với những so sánh cho thấy độ chính xác tương đương YOLOv4 và có tốc độ nhanh hơn khi thực hiện dự đoán. Khác với những phiên bản tiền nhiệm, YOLOv5 được phát triển dựa trên PyTorch thay vì DarkNet. Đây là một điểm cộng không nhỏ cho YOLOv5 vì PyTorch phổ biến hơn rất nhiều, điều này đồng nghĩa với việc sẽ có nhiều tài liệu và hướng dẫn cho chúng ta tham khảo về mô hình này. Có thể thấy cấu trúc YOLOv5 không có nhiều thay đổi quá lớn nhưng lại được phát triển trên PyTorch và có hướng dẫn rất chi tiết và đầy đủ. Do đó, YOLOv5 được sử dụng cho đề tài này.

Về kiến trúc, mạng YOLO bao gồm: Base network là các mạng tích chập có nhiệm vụ trích xuất đặc trưng. Tiếp theo sau là những Extra Layers được áp dụng để phát hiện vật thể trên bản đồ đặc tính của base network.

Về phương thức thuật toán YOLO hoạt động bằng cách sử dụng ba kỹ thuật sau:

- *Residual blocks (khối dư)*
- *Bounding box regression (hồi quy hộp giới hạn).*
- *Intersection Over Union (IOU).*

Đầu vào của mô hình là một ảnh, mô hình sẽ nhận dạng ảnh đó có đối tượng nào hay không, sau đó sẽ xác định tọa độ của đối tượng trong bức ảnh. Ảnh đầu vào được chia thành AxA ô thường thì sẽ là 3x3, 5x5, 7x7, ... Việc chia lưới này sẽ có ảnh hưởng không nhỏ tới mô hình phát hiện đối tượng.



Hình 3. 7 Ví dụ về chia lưới và vẽ đường bao cho vật thể

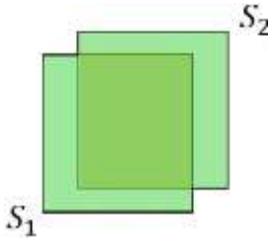
Với input là 1 ảnh, đầu ra mô hình là một ma trận 3 chiều có kích thước A x A x (5 x N + M) với số lượng tham số mỗi ô là (5 x N + M) với N và M lần lượt là số lượng Box và Class mà mỗi ô cần dự đoán. Ví dụ với hình ảnh trên chia thành 7 x 7 ô, mỗi ô cần dự đoán 2 bounding box và 2 vật thể: smoke và fire (lửa và khói). Như vậy output sẽ là 7 x 7 x 12, mỗi ô sẽ có 12 tham số, kết quả trả về là 98 bounding box. Vậy thông số (5 x N + M) được hình thành như sau:

*Dự đoán rằng mỗi bounding box gồm 5 thành phần là tọa độ (x, y) của tâm vòng bao, sở dĩ ta chọn A x A là số lẻ bởi cần xác định pixel là tâm của vòng bao, (h, w) lần lượt là chiều cao và chiều rộng của bounding box, prediction (dự đoán): Pr (Object) * IOU (Pred, Truth). Như giả định ở phần trên, có 12 tham số, có thể hiểu đơn giản rằng tham số thứ 1 sẽ chỉ ra ô đó có đối tượng hay không, tham số 2,3,4,5 sẽ trả về x, y, h, w của Box1. Tham số 6,7,8,9,10 tương tự sẽ cho Box2, tham số 11,12 lần lượt là xác suất ô đó có chứa object1(lửa), object2(khói). Lưu ý rằng tâm của bounding box nằm ở ô nào thì ô đó sẽ chứa đối tượng, cho dù đối tượng có thể ở các ô khác thì cũng sẽ trả về là 0. Vì vậy việc mà 1 ô chứa 2 hay nhiều tâm của bouding box hay đối tượng thì sẽ không thể detect được, đó là một hạn chế của mô hình YOLO1, vậy ta cần phải tăng số lượng ô chia trong 1 ảnh lên đó là lí do vì sao mình nói việc chia ô có thể làm ảnh hưởng tới việc mô hình phát hiện đối tượng.*

Hàm tính IOU, trong thuật toán này, sử dụng theo dõi đối tượng kết hợp với YOLOV5. Theo dõi đối tượng là nhiệm vụ tự động xác định các đối tượng trong video và tính toán quỹ đạo của từng đối tượng. Theo dõi đối tượng thường được sử dụng trong các lĩnh vực giám sát như nhận dạng biển số tự động.

Qua đó, chúng ta có thể thấy sự khác biệt với các thuật toán phát hiện đối tượng thông thường là YOLO không phân biệt các đối tượng cùng lớp còn YOLOV5+ theo dõi đối tượng có mỗi đối tượng được cấp một id duy nhất. Các phát hiện đối tượng trong các khung hình liên tiếp có tỷ lệ trùng lặp cao. Dưới đây là công thức nhằm đánh giá độ chính xác của object detector trên tập dữ liệu cụ thể:

$$IOU\ \ S_1, S_2\ = \frac{S_1 \cap S_2}{S_1 \cup S_2} \qquad (3.11)$$



$$D_0, D_1, ..., D_{F-1}\ = \left\{ \begin{matrix} d_0, d_1, ..., d_{N-1}\ ,\ d_0, d_1, ..., d_{N-1}\ , \\ ...\ d_0, d_1, ..., d_{N-1} \end{matrix} \right\} \qquad (3.12)$$

D_f : *Nhận diện tại khung hình thứ f*

d_f : *Đối tượng được nhận diện tại khung hình thứ f.*

Nếu IoU > 0.5 thì prediction được đánh giá là tốt.

YOLOv5 có sử dụng 3 đầu ra từ PAN Neck, để phát hiện objects tại 3 scale khác nhau. Tuy nhiên, Glenn Jocher nhận thấy rằng sự ảnh hưởng của các object tại mỗi scale đến Objectness Loss là khác nhau, do đó, công thức của Objectness Loss được thay đổi như sau:

$$L_{classification} = \sum_{i=0}^{S^2} II_i^{obj} \sum_{c \in class} \left(p_i(c) - \hat{p}_i(c) \right)^2 \qquad (3.13)$$

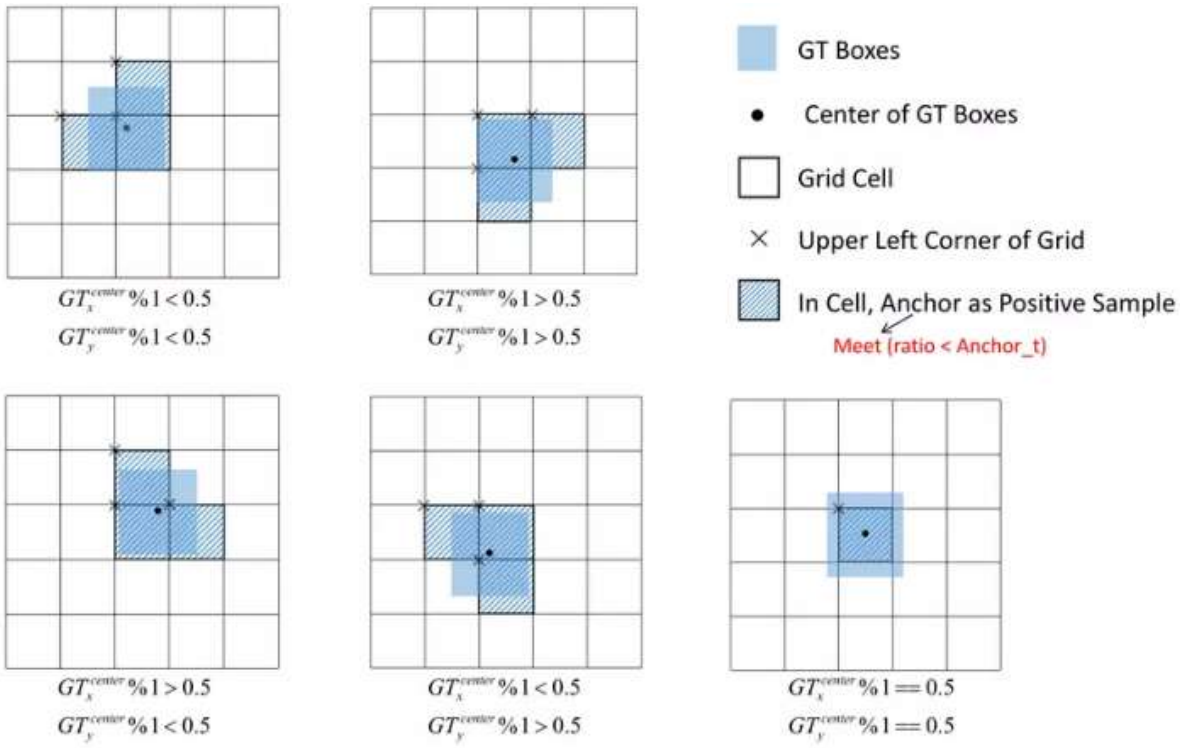
Trong đó:

II_i^{obj} *: bằng 1 nếu ô vuông đang xét có object ngược lại bằng 0*

$\hat{p}_i(c)$ *: là xác xuất có điều của lớp c tại ô vuông tương ứng mà mô hình dự đoán*

Anchor Box trong YOLOv5 nhận được 2 sự thay đổi lớn so với các phiên bản tiền nhiệm. Đầu tiên là sử dụng Auto Anchor, một kĩ thuật áp dụng giải thuật di truyền (GA) vào Anchor Box ở sau bước k-means để Anchor Box hoạt động tốt hơn với những custom dataset của người dùng, chứ không còn hoạt động tốt ở mỗi COCO nữa [6].

Thứ hai đó chính là offset tâm của object để lựa chọn nhiều Anchor Box cho một object. Ý tưởng được thể hiện trong hình dưới:



Hình 3. 8 Anchor Box cho một object.

3.3.2. Chuẩn bị và tiền xử lý dữ liệu

Chuẩn bị dữ liệu là quá trình chuẩn hóa và biến đổi dữ liệu thô trước khi xử lý và phân tích. Đây là một bước quan trọng trước khi xử lý và thường liên quan đến việc định dạng lại dữ liệu, chỉnh sửa dữ liệu và kết hợp các bộ dữ liệu để làm phong phú dữ liệu.

Chuẩn bị dữ liệu thường là một công việc mất nhiều thời gian đối với các kỹ sư dữ liệu hoặc người dùng doanh nghiệp, nhưng điều cần thiết là điều kiện tiên quyết là đưa dữ liệu vào ngữ cảnh để biến dữ liệu thành thông tin chuyên sâu và loại bỏ sai lệch do chất lượng dữ liệu kém.

Ví dụ: quy trình chuẩn bị dữ liệu thường bao gồm chuẩn hóa định dạng dữ liệu, làm phong phú dữ liệu nguồn và/hoặc loại bỏ các giá trị ngoại lai.

Quá trình chuẩn bị dữ liệu bắt đầu với việc tìm đúng dữ liệu. Điều này có thể đến từ một danh mục dữ liệu hiện có hoặc nguồn dữ liệu có thể được thêm vào đặc biệt.

Do các kịch bản phức tạp và thông tin tính năng hạn chế trong một hình ảnh duy nhất, việc phát hiện khói chính xác khó khăn hơn nhiều trong thực tế. Một vấn đề quan trọng là làm thế nào để xây dựng một tập dữ liệu huấn luyện gồm các hình ảnh khói được ghép nối và các vị trí hộp giới hạn sự thật cơ bản để học từ đầu đến cuối. Với bộ

dữ liệu được xây dựng, kết quả thử nghiệm chứng minh rằng các mô hình phân biệt đối xử có thể được đào tạo một cách hiệu quả như máy dò khói để phát hiện chính xác các đám cháy âm ỉ. Bộ dữ liệu về khói từ Smoke 100k: A Database for Smoke Detection được sử dụng với các tùy chọn như khói cao, khói thấp và khói trung bình. Về dữ liệu hỏa hoạn, chúng tôi thu thập trên internet các video về hỏa hoạn, ngọn lửa, v.v ...

Theo truyền thống, dọn dẹp dữ liệu là phần tốn nhiều thời gian nhất của quy trình chuẩn bị dữ liệu, nhưng nó rất quan trọng để xóa dữ liệu bị lỗi và lấp đầy các khoảng trống. Nhiệm vụ quan trọng ở đây bao gồm:

- Loại bỏ dữ liệu không liên quan và ngoại lệ.

- Điền các giá trị còn thiếu.

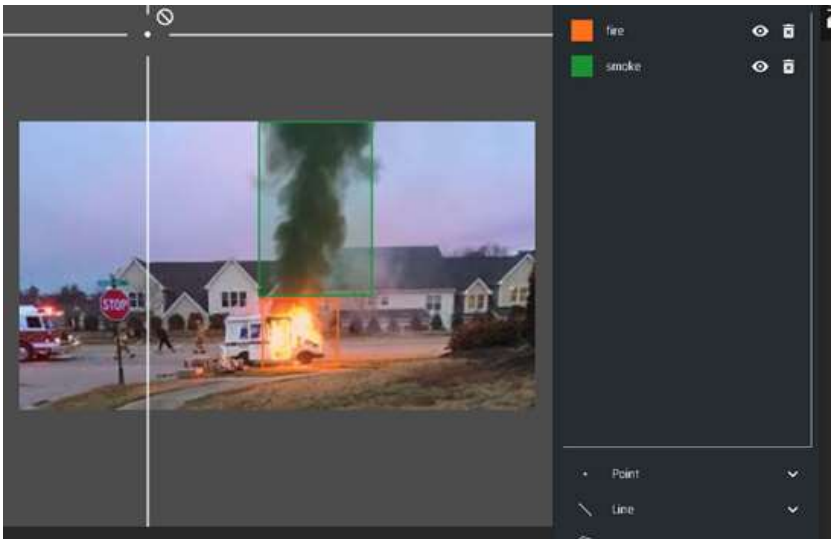
- Tuân thủ dữ liệu theo một mẫu chuẩn hóa.

- Xóa các mục nhập dữ liệu riêng tư hoặc nhạy cảm.

Khi dữ liệu đã được làm sạch, nó phải được xác thực bằng cách kiểm tra lỗi trong quá trình chuẩn bị dữ liệu cho đến thời điểm này. Thông thường, một lỗi trong hệ thống sẽ trở nên rõ ràng trong bước xác thực này và sẽ cần được giải quyết trước khi tiếp tục.

Tất cả dữ liệu thu thập được chia thành 3 loại tương ứng với các tệp trong ổ đĩa: thấp, trung bình và cao đối với khói và tương tự đối với lửa. Nhưng để đơn giản trong lập trình, chúng ta chỉ sử dụng 2 nhãn là lửa và khói.

Dữ liệu thu thập được sau đó được dán nhãn từ nền tảng trực tuyến makesense.ai. Như thể hiện trong Hình dưới đây, nhãn được tạo dưới dạng ba đối tượng: người, ô tô và thông báo và chú thích, và hình ảnh chú thích được chuyển đổi thành định dạng văn bản theo định dạng YOLO.



Hình 3. 9 Gắn nhãn và xuất đặc trưng theo định dạng cua YOLO

Tệp cấu hình dữ liệu mô tả các tham số của bộ dữ liệu. Do đang đào tạo về bộ dữ liệu lửa và khói tùy chỉnh nên cần chỉnh sửa tệp này và cung cấp: đường dẫn đến bộ dữ liệu đào tạo, xác thực và kiểm tra (tùy chọn); số lớp (nc); và tên của các lớp theo cùng thứ tự như chỉ mục của chúng với hai lớp là 'smoke' và 'fire'. Tệp cấu hình dữ liệu tùy chỉnh 'smokeandfire.yaml' trong thư mục 'data'. Nội dung của tệp YAML này như sau:

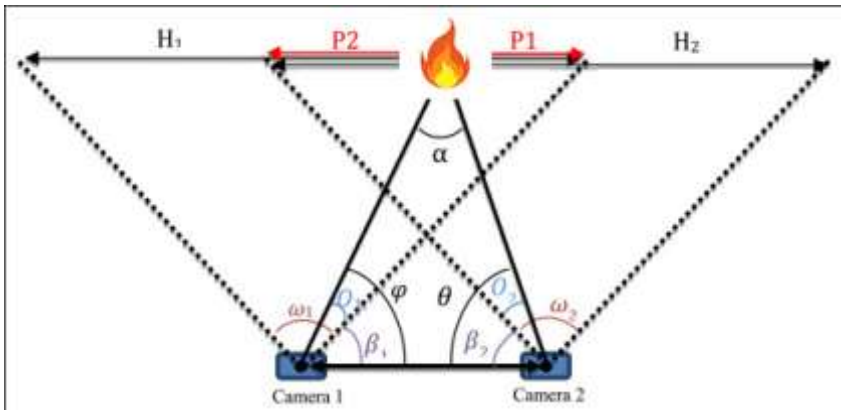
```
train: ../train_data/images/train/
val: ../train_data/images/val/


# Classes
nc: 2  # number of classes
names: ['smoke','fire']  # class names
```

Hình 3. 10 Tệp cấu hình của bộ dữ liệu.

3.3.3. Thuật toán xác định vị trí sau khi phát hiện nguy cơ cháy

Sau khi nhận diện được khung hình nơi có điểm hỏa hoạn, nhiệm vụ tiếp theo của thiết bị chính là đo đạc và tính toán khoảng cách của điểm lửa đối trong thời gian thực. Ý tưởng chính là ghi lại một cảnh từ hai điểm khác nhau và sử dụng sự chênh lệch để chỉ ra vị trí, mối quan hệ và cấu trúc của các vật thể trong cảnh. Sự khác biệt giữa các vị trí điểm ảnh trong hai ảnh tạo ra ấn tượng về độ sâu. Khoảng cách của vật thể được đo khi nó ở trong trường nhìn chồng chéo của hai máy ảnh [7].



Hình 3. 11 Hệ thống camera kép sử dụng trong tính toán khoảng cách

Như trong Hình hai máy ảnh được gắn theo chiều ngang và cách nhau theo khoảng cách A. trong đó h là khoảng cách mong muốn giữa đối tượng và máy ảnh. Để đo khoảng cách h, chúng ta cần các tham số sau:

B: khoảng cách tách đối tượng và camera bên trái.

C: khoảng cách tách đối tượng và máy ảnh bên phải.

α , θ ,ϕ *là các góc của hình tam giác được hình thành bởi đối tượng và hai máy ảnh*

Theo công thức lượng giác:

$$\sin\varphi = \frac{h}{B}$$

$$\sin\theta = \frac{h}{C}$$

Do đó $h = B.\sin\varphi = C\sin\theta$

Lại có: $\frac{A}{\sin\sin\alpha} = \frac{B}{\sin\sin\theta}$ *.nên* $B = \frac{A\sin\sin\theta}{\sin\sin\alpha}$

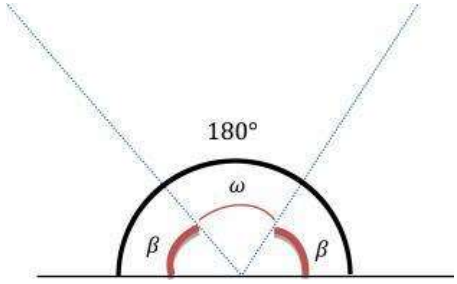
Ta có công thức tính được khoảng cách h:

$$h = \frac{A\sin\sin\theta\sin\sin\phi}{\sin\sin\alpha} \qquad (3.14)$$

w1, w2*: Góc nhìn tương ứng của 2 máy ảnh.*

H_1, H_2: *Số điểm ảnh ngang của 2 máy ảnh.*

P_1, P_2*: vị trí của vật thể trong cả hai máy ảnh, trong đó* P_1 *là khoảng cách về điểm ảnh giữa centroid của vật thể và cuối vùng chồng chéo cho máy ảnh ở bên trái.*



Hình 3. 12 Các góc của máy ảnh

Vậy khoảng cách h được tính như sau:

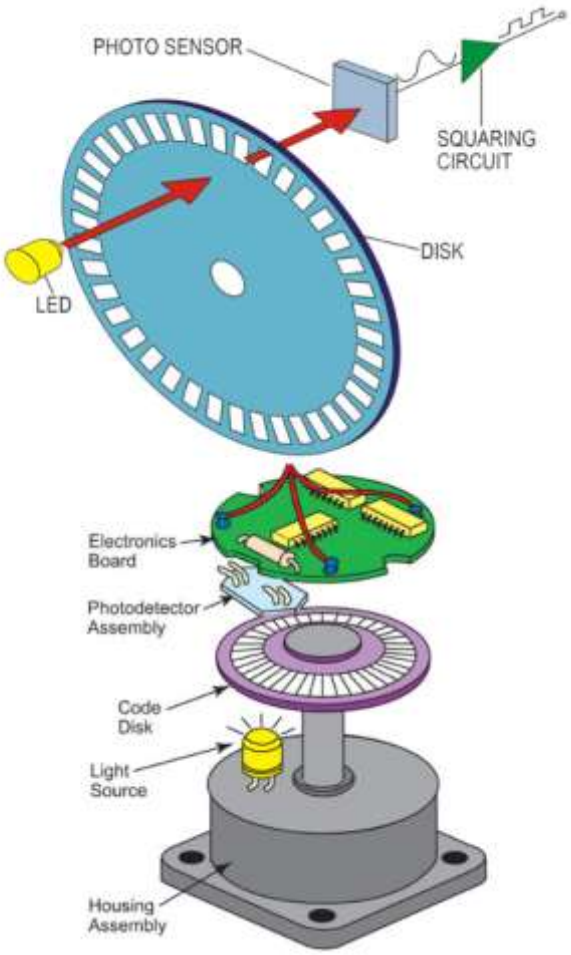
$$h = \frac{A\sin\left(P_2.\frac{w_2}{H_2} + \beta_2\right)\sin\left(P_1.\frac{w_1}{H_1} + \beta_1\right)}{\sin\left(180 - \left(P_1.\frac{w_1}{H_1} + \beta_1 + P_2.\frac{w_2}{H_2} + \beta_2\right)\right)} \qquad (3.15)$$

CHƯƠNG 4: ĐIỀU KHIỂN VÀ THỰC NGHIỆM

4.1. Điều khiển robot sử dụng 3 bánh Omni

Robot sử dụng vi điều khiển, động cơ DC JGB37 tích hợp encoder kết hợp với mạch cầu H để điều khiển và truyền động. Do robot sử dụng 3 bánh Omni có độ linh hoạt cao nên cần nắm được được vận tốc và quãng đường tại mỗi bánh xe để robot hoạt động đúng theo tính toán động học và động lực học. Vì vậy nên sử dụng bộ điều khiển PID để hiệu chỉnh là cần thiết đối với robot.

4.1.1. Thu thập tín hiệu encoder để và sử bộ điều khiển PID để tính toán công suất cho động cơ

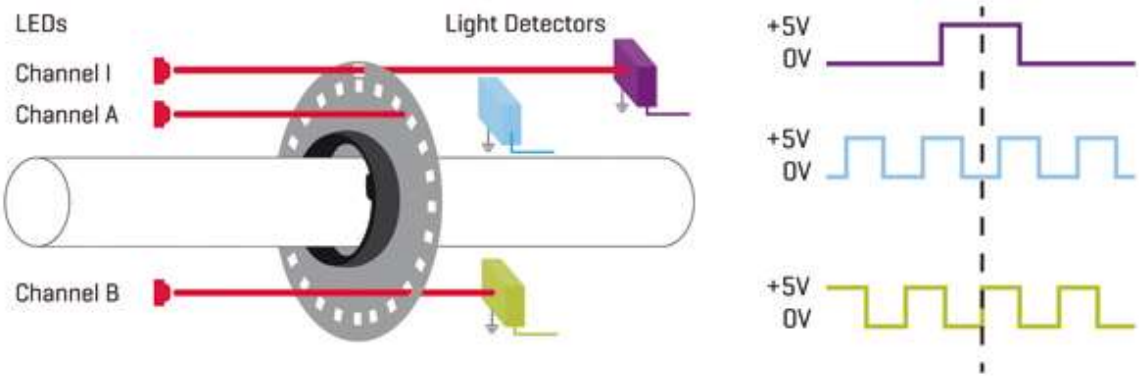


Hình 4. 1 Cấu tạo của encoder

Tín hiệu từ encoder tạo ra các dạng xung vuông có tần số thay đôi phụ thuộc vào tốc độ động cơ. Do đó các xung vuông này được đưa vào bộ vi xử lý để đếm số xung trong khoảng thời gian cho phép từ đó ta có thể tính được giá trị vận tốc của động cơ.

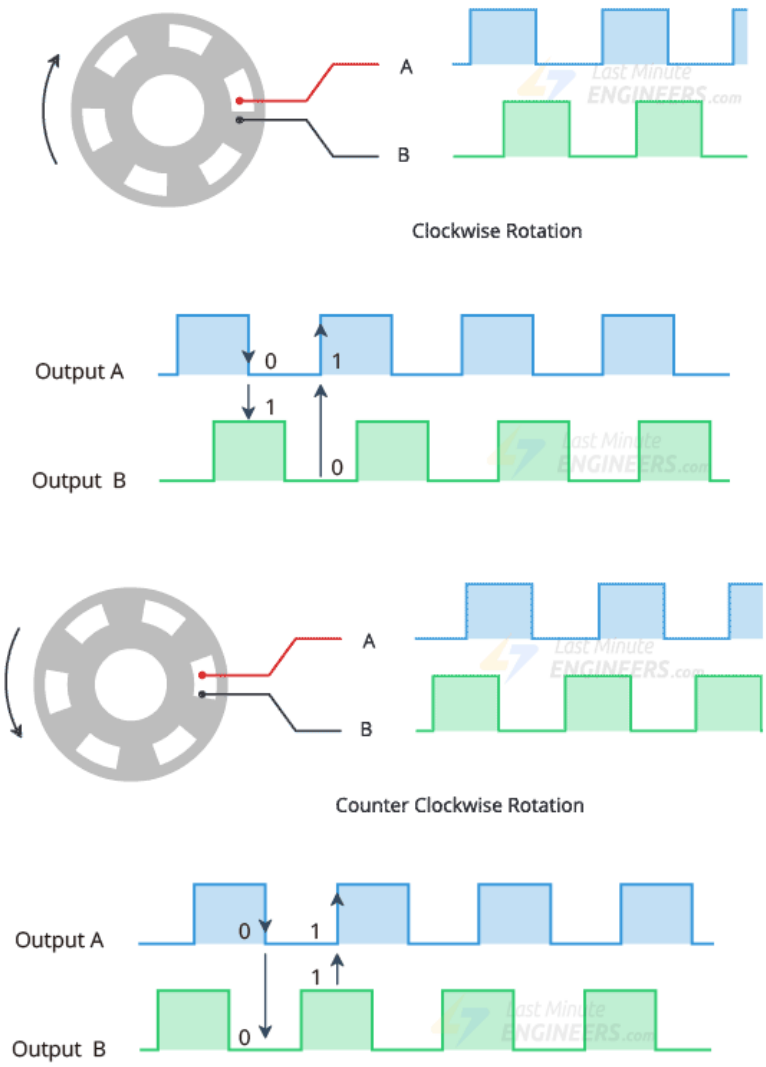
Nguyên lý cơ bản của encoder, đó là một đĩa tròn xoay, quay quanh trục. Trên đĩa có các lỗ (rãnh). Người ta dùng một đèn led để chiếu lên mặt đĩa. Khi đĩa quay, chỗ không có lỗ (rãnh), đèn led không chiếu xuyên qua được, chỗ có lỗ (rãnh), đèn led sẽ chiếu xuyên qua. Khi đó, phía mặt bên kia của đĩa, người ta đặt một con mắt thu. Với các tín hiệu có, hoặc không có ánh sáng chiếu qua, người ta ghi nhận được đèn led có chiếu qua lỗ hay không. Số xung đếm được và tăng lên nó tính bằng số lần ánh sáng bị cắt.

Như vậy là encoder sẽ tạo ra các tín hiệu xung vuông và các tín hiệu xung vuông này được cắt từ ánh sáng xuyên qua lỗ. Nên tần số của xung đầu ra sẽ phụ thuộc vào tốc độ quay của tấm tròn đó.



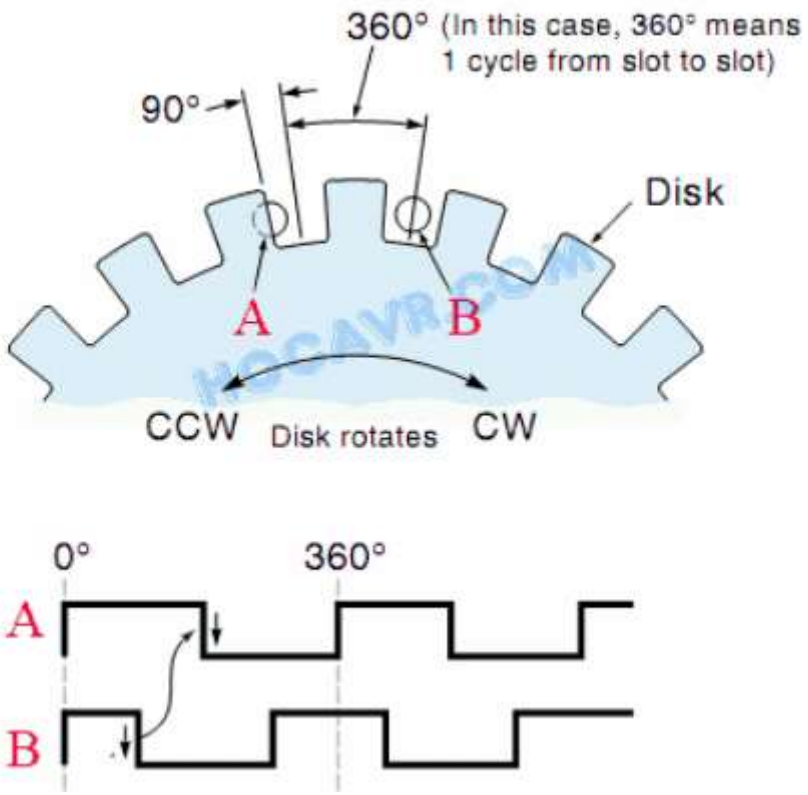
Hình 4. 2 Tín hiệu của encoder khi hoạt động

Encoder được tích hợp trong động cơ JGB37 có nhiều ưu điểm, bao gồm khả năng đo vị trí quay chính xác, khả năng đo tốc độ quay và độ ổn định cao. Ngoài ra, việc tích hợp encoder vào động cơ JGB37 cũng giúp tiết kiệm được không gian, giảm thiểu sự phức tạp trong thiết kế và lắp đặt của hệ thống điều khiển động cơ. Encoder mà động cơ JGB37 sử dụng là loại encoder tương đối có 3 kênh (3 ngõ ra) bao gồm kênh A, kênh B và kênh I (Index). Trong hình trên có thể thấy một lỗ nhỏ bên phía trong của đĩa quay và một cặp phát-thu dành riêng cho lỗ nhỏ này. Đó là kênh I của encoder. Cứ mỗi lần motor quay được một vòng, lỗ nhỏ xuất hiện tại vị trí của cặp phát-thu, hồng ngoại từ nguồn phát sẽ xuyên qua lỗ nhỏ đến cảm biến quang, một tín hiệu xuất hiện trên cảm biến. Như thế kênh I xuất hiện một "xung" mỗi vòng quay của motor.



Hình 4. 3 Tín hiệu encoder khi motor quay cùng và ngược chiều kim đồng hồ

Bên ngoài đĩa quay được chia thành các rãnh nhỏ và một cặp thu-phát khác dành cho các rãnh này. Đây là kênh A của encoder, hoạt động của kênh A cũng tương tự kênh I, điểm khác nhau là trong 1 vòng quay của motor, có N "xung" xuất hiện trên kênh A. N là số rãnh trên đĩa và được gọi là độ phân giải (resolution) của encoder. Mỗi loại encoder có độ phân giải khác nhau, có khi trên mỗi đĩa chỉ có vài rãnh nhưng cũng có trường hợp đến hàng nghìn rãnh được chia. Để điều khiển động cơ cần biết độ phân giải của encoder đang dùng. Độ phân giải ảnh hưởng đến độ chính xác điều khiển và cả phương pháp điều khiển. Trên các encoder còn có một cặp thu phát khác được đặt trên cùng đường tròn với kênh A nhưng lệch một chút, đây là kênh B của encoder. Với 2 tín hiệu xung A và B giúp chúng ta xác định chiều quay của động cơ. Tín hiệu xung từ kênh B có cùng tần số với kênh A nhưng lệch pha 90 độ [7].



Hình 4. 4 Xung A và B lệch pha nhau 90 độ

Với tín hiệu encoder trả về, ta hoàn toàn có thể xây dựng bộ điều khiển PID cho từng động cơ, bộ điều khiển PID có nhiệm vụ đưa sai lệch e(t) của hệ thống về 0 sao cho quá trình quá độ thỏa mãn các yêu cầu cơ bản về chất lượng:

- *Nếu sai lệch e(t) càng lớn thì thông qua khâu khuếch đại, tín hiệu u(t) càng lớn.*
- *Nếu sai lệch e(t) chưa bằng 0 thì thông qua khâu tích phân, PID vẫn còn.*
- *tạo tín hiệu điều chỉnh.*
- *Nếu thay đổi của sai lệch e(t) càng lớn thì thông qua thành phần vi phân, phản ứng thích hợp của u(t) sẽ càng nhanh.*

Bộ điều khiển PID được mô tả bằng mô hình vào-ra

$$U(t) = K_p\,(e(t) + \frac{1}{Ti}\int_0^t e(\tau)\,d\tau) + Td\,\frac{de(t)}{dt}Ti\frac{1}{Ti} \qquad (4.1)$$

Ta cần khởi tạo và đặt thời gian lấy mẫu "T", khi lập trình, nếu khoảng thời gian chạy chương trình lớn hơn hoặc bằng giá trị lấy mẫu, ta sẽ tính toán được số xung chạy trong khoảng thời gian "T". Với số xung tính được từ 3 bánh xe, vận tốc của 3 bánh xe được tính dựa trên số xung đếm được trong khoảng thời gian "T", số xung trên mỗi vòng của encoder, bánh kính bánh xe và số xung tại thời điểm trước đó của chu kì "T".

Sau khi đã tính toán được vận tốc của từng bánh xe sau mỗi thời gian lấy mẫu "T", sai số giữa vận tốc tức thời và vận tốc mong muốn được tính bởi công thức:

$$E = ve - vr \quad (4.2)$$

Trong đó:

E là sai số vận tốc cần tính toán

ve là tốc độ mong muốn của 3 bánh xe

vr là tốc độ thực tế của từng bánh xe

- *Xây dựng bộ điều khiển PID và tính toán xung xuất ra cho từng động cơ*

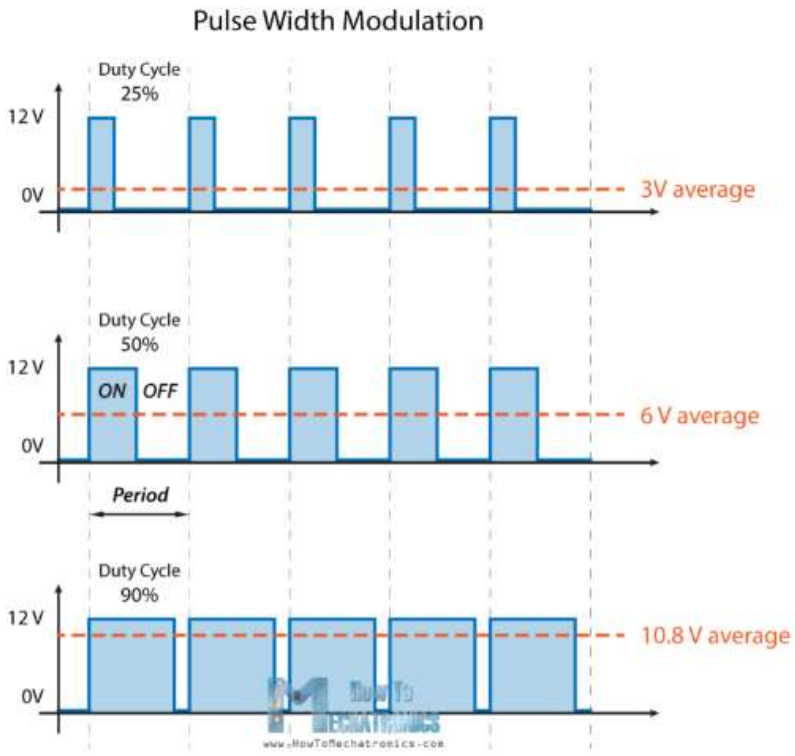
Việc hiệu chỉnh sai số cho động cơ yêu cầu bộ điều khiển PID có bộ thông số phù hợp, trong phần lập trình này, bộ điều khiển PID sẽ trả về giá trị của tín hiệu analog xuất ra, từ đó hiệu chỉnh được tốc độ bánh xe.

```
int PID(float E, float E1, int pulse_of_motor_before)
{
  float delta = kp*E  + kd*E1;
  float pulse_of_motor = round(pulse_of_motor_before + delta);
  if(pulse_of_motor > 255)
  {
    pulse_of_motor = 255;
  }
  E1 = E;
  return pulse_of_motor;
}
```

Hình 4. 5 Lập trình bộ điều khiển PID tính toán độ rộng xung đầu ra cho mỗi motor

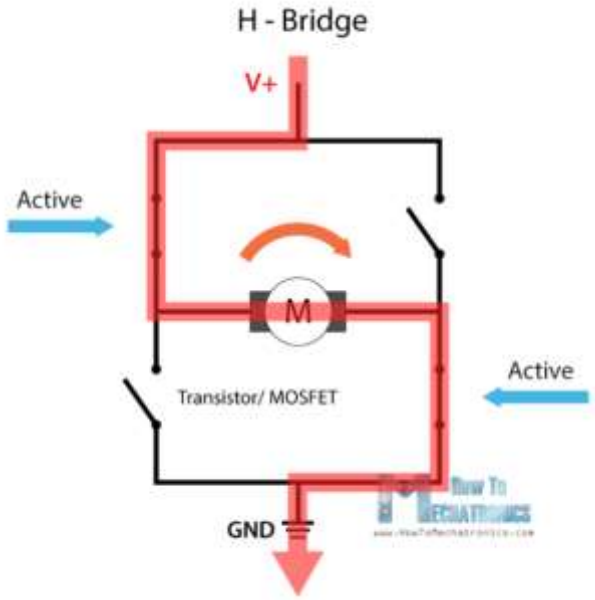
4.1.2. Điều khiển động cơ bằng phương pháp điều chế độ rộng xung

Để điều khiển từng động cơ với công suất cao ta cần kết hợp giữa mạch cầu H và vi điều khiển, với robot 3 bánh Omni, vi điều khiển được sử dụng là arduino Mega2560 do vi điều khiển này có đủ số chân ngắt và đủ khả năng tính toán và điều khiển 3 động cơ DC. Phương pháp điều khiển là điều chế độ rộng xung (PWM). PWM hay điều chế độ rộng xung là một kỹ thuật cho phép chúng ta điều chỉnh giá trị trung bình của điện áp đến thiết bị điện tử bằng cách bật và tắt nguồn với tốc độ nhanh. Điện áp trung bình phụ thuộc vào chu kỳ xung, hoặc lượng thời gian tín hiệu BẬT so với lượng thời gian tín hiệu TẮT trong một khoảng thời gian quy định.



Hình 4. 6 Phương pháp điều chế độ rộng xung

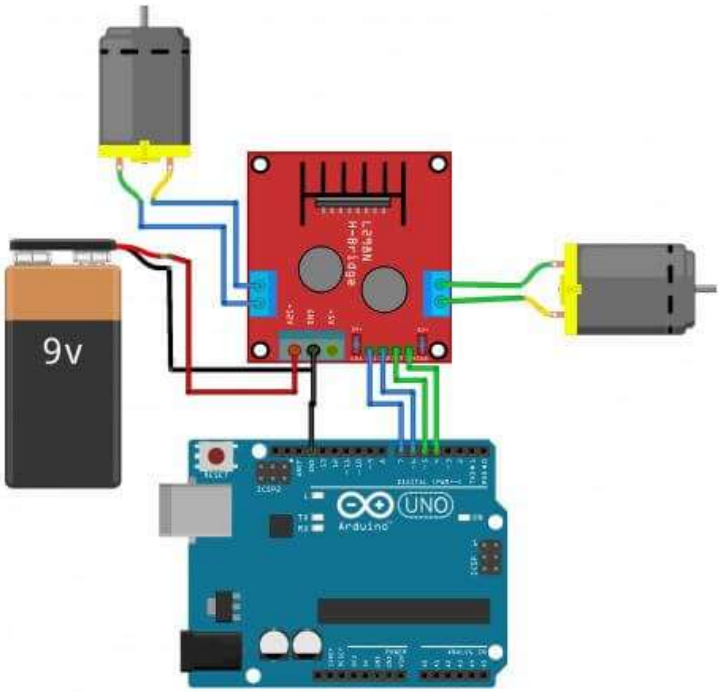
Kết hợp cùng với mạch cầu H, ta có thể điều khiển công suất trên mỗi động cơ và chiều xoay của mỗi động cơ.



Hình 4. 7 Mạch cầu H sử dụng MOSFET

Mỗi một mạch cầu H có thể điều khiển được 2 động cơ, các chân Đầu vào 1 và Đầu vào 2 được sử dụng để điều khiển hướng quay của động cơ A và đầu vào 3 và 4

cho động cơ B. Sử dụng các chân này để điều khiển các công tắc của mạch cầu H. Nếu đầu vào 1 ở mức THẤP và đầu vào 2 là CAO thì động cơ sẽ di chuyển về phía trước và ngược lại, nếu đầu vào 1 ở mức CAO và đầu vào 2 ở mức THẤP thì động cơ sẽ di chuyển lùi. Trong trường hợp cả hai đầu vào đều giống nhau, cùng THẤP hoặc CAO, động cơ sẽ dừng. Điều tương tự áp dụng cho đầu vào 3 và 4 và động cơ B [8].



Hình 4. 8 Sơ đồ kết nối giữa vi điều khiển, mạch cầu H và 2 động cơ DC

Với việc đã tính toán được tín hiệu đầu ra dựa vào bộ điều khiển PID, robot sẽ khởi tạo thời gian lấy mẫu "T". Với số xung tính được từ 3 bánh xe, vận tốc của 3 bánh xe được tính dựa trên số xung đếm được trong khoảng thời gian "T", số xung trên mỗi vòng của encoder, bánh kính bánh xe và số xung tại thời điểm trước đó của chu kì "T".

Sau mỗi một chu kì "T", công suất mới của động cơ sẽ được cung cấp bởi tín hiệu PWM giúp động cơ được cung cấp năng lượng thích hợp từ mạch cầu H và nguồn, từ đó giúp đảm bảo động cơ có thể quay với tốc độ đặt ra.

```
if(PID_time_a - PID_time_b >= T )
{
  pulse1a = count1;
  pulse2a = count2;
  pulse3a = count3;
  float vr1 = (pulse1a - pulse1)*408.2*1000/(400*T);
  float vr2 = (pulse2a - pulse2)*408.2*1000/(400*T);
  float vr3 = (pulse3a - pulse3)*408.2*1000/(400*T);
  E1a = ve - vr1;
  E2a = ve - vr2;
  E3a = ve - vr3;
  pulse_of_motor1 = PID(E1a,E1b,pulse_of_motor1);
  pulse_of_motor2 = PID(E2a,E2b,pulse_of_motor2);
  pulse_of_motor3 = PID(E3a,E3b,pulse_of_motor3);
  PID_time_b = PID_time_a;
}
```

Hình 4. 9 Lập trình tính toán và điều khiển động cơ dựa trên bộ điều khiển PID

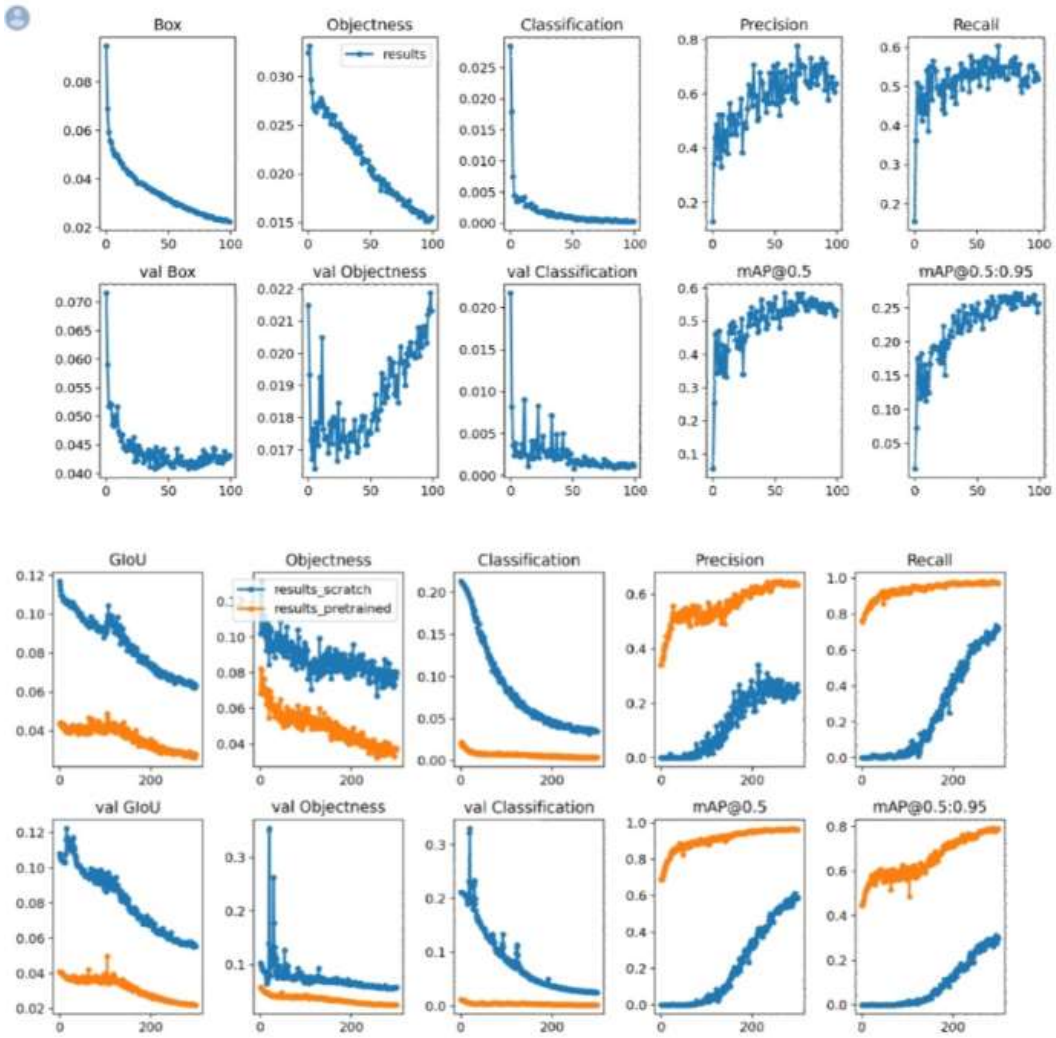
4.2. Thực nghiệm với robot sử dụng 3 bánh Omni



Hình 4. 10 Robot chạy thực nghiệm

Robot được chạy thực nghiệm với quỹ đạo được quy định sẵn. Kết quả thực nghiệm cho thấy robot hoạt động đúng với những phương trình động học và động lực học được đề xuất, bộ điều khiển PID hoạt động tốt giúp robot đảm bảo sai số tốc độ nhỏ nhất ở mỗi bánh. Bánh Omni linh hoạt giúp robot di chuyển đa hướng trong không gian, linh hoạt di chuyển đến vị trí tọa độ mong muốn.

Với mô hình nhận dạng lửa, việc huấn luyện mô hình trí tuệ nhân tạo cho ra kết quả như sau:



Hình 4. 11 Kết quả huấn luyện mô hình

Đồ thị kết quả YOLOv5 cho lửa và khói cho thấy một độ chính xác tốt trong việc phát hiện và phân loại các thực thể là lửa và khói. Nó cũng cho thấy rằng mô hình không bị overfit, điều này có thể được giải thích bằng sự đồng nhất giữa các giá trị của đường cong training và validation.

Cụ thể, đồ thị cho thấy rằng độ chính xác của mô hình đạt tới hơn 93% khi phát hiện và phân loại lửa và khói. Điều này cho thấy rằng YOLOv5 là một phương pháp hiệu quả để giúp mọi người phát hiện và phân loại lửa và khói.

Trong đồ thị YOLOv5, độ chính xác và độ phủ là hai độ đo quan trọng để đánh giá hiệu suất của mô hình nhận dạng. Độ chính xác là tỷ lệ giữa số lượng các đối tượng đã được phát hiện chính xác và tổng số lượng đối tượng trong tập dữ liệu. Trong khi đó, độ phủ

là tỷ lệ giữa số lượng các đối tượng đã được phát hiện chính xác và tổng số lượng các đối tượng có thể được phát hiện.

Một mô hình YOLOv5 với độ chính xác cao sẽ có xu hướng phát hiện đúng nhiều đối tượng hơn so với mô hình có độ chính xác thấp. Tuy nhiên, việc tăng độ chính xác có thể làm giảm độ phủ của mô hình. Điều này có nghĩa là mô hình có thể bỏ sót một số đối tượng vì ngưỡng phát hiện được đặt quá cao. Do đó, trong đồ thị YOLOv5, cần thiết phải tìm ra sự cân bằng giữa độ chính xác và độ phủ để đạt được hiệu suất tốt nhất trong việc nhận dạng đối tượng. Có thể thấy đồ thị về độ chính xác (Precision) và độ phủ (Recall) khá cân bằng với nhau, vì vậy có thể nhận định rằng hệ thống cơ bản đáp ứng được độ chính xác cần thiết và không bị quá khớp dữ liệu (overfit) và xác suất đầu ra đáp ứng được mục đích đánh giá và cảnh báo sớm hỏa hoạn. Dưới đây là một số hình ảnh về kết quả thử nghiệm trên tập ảnh thử nghiệm:



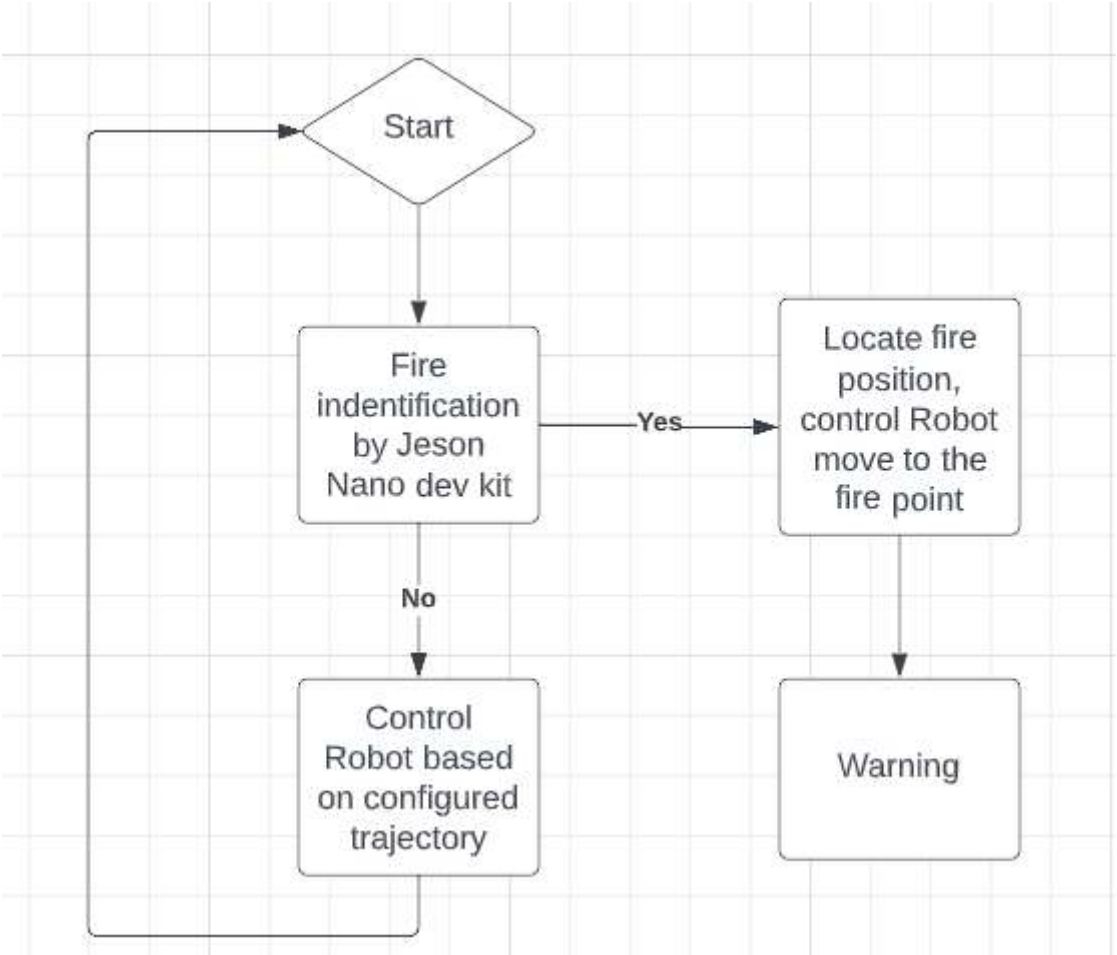
Hình 4. 12 Kết quả thử nghiệm mô hình nhận dạng



Hình 4. 13 Kết quả thử nghiệm mô hình nhận dạng

Sau khi đã đánh giá được độ chính xác của mô hình với tập kiểm thử, mô hình trí tuệ nhân tạo được ứng dụng trên quá trình thực nghiệm. Mục đích của công việc này là

thử nghiệm khả năng nhận dạng của robot trong môi trường và hình ảnh thực tế. Nhóm đã thực hiện kết nối máy tính nhúng Jetson Nano đã được triển khai mô hình nhận dạng cháy với vi điều khiển trên robot. Khi đó máy tính nhúng Jetson Nano sẽ được sử dụng để giám sát và tính toán nguy cơ cháy từ hình ảnh trả về, sau đó truyền nhận thông tin với vi điều khiển để điều hướng. Quá trình thực nghiệm sẽ diễn ra theo sơ đồ sau:

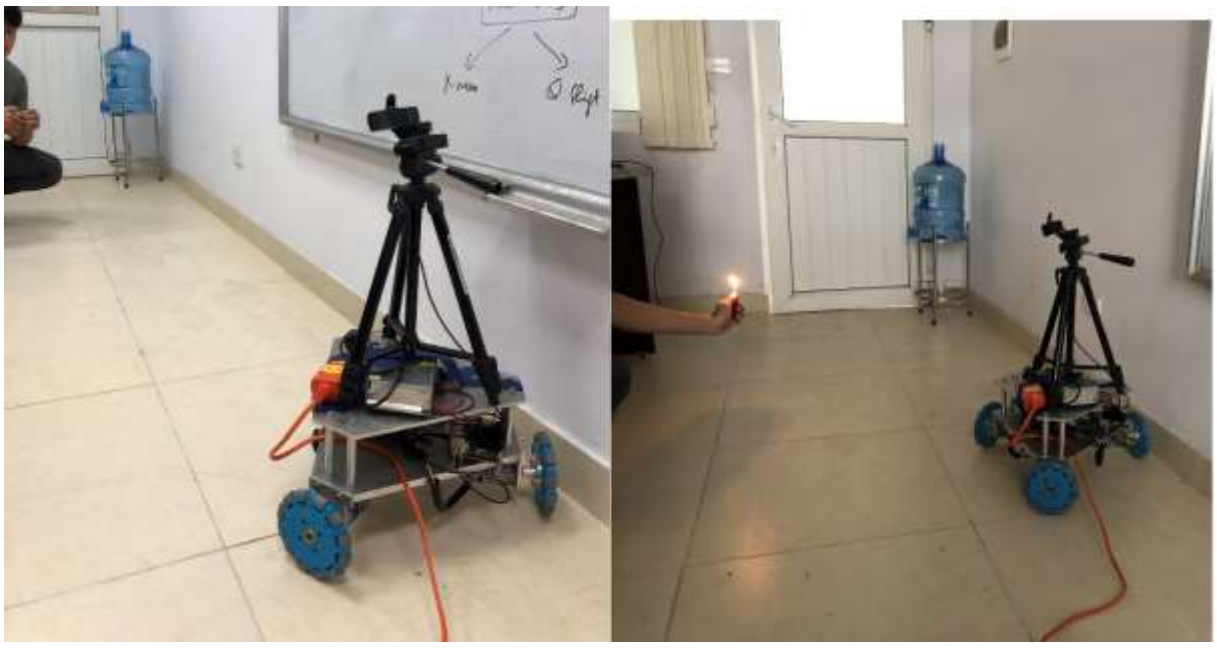


Hình 4. 14 Sơ đồ khối của mô hình Robot

Theo sơ đồ khối có thể thấy khi không xác định được nguy cơ cháy, robot sẽ di chuyển theo quỹ đạo được quy định trước trong phạm vi hoạt động, khi có nguy cơ cháy xảy ra, máy tính nhúng Jetson Nano sẽ đóng vai trò quan trọng trong việc nhận dạng và đưa ra quỹ đạo di chuyển phù hợp cho robot. Quá trình thực nghiệm nhận dạng lửa trên robot sử dụng Jetson Nano đã cho ra được kết quả về nguy cơ cháy, vị trí và tọa độ của nguồn cháy, từ đó giúp robot thay đổi quỹ đạo chuyển động và di chuyển đến tọa độ cháy để đưa ra cảnh báo.



Hình 4. 15 Thực nghiệm nhận dạng và cảnh báo nguy cơ cháy trên robot



Hình 4. 16 Robot thực hiện điều hướng theo hướng của nguy cơ cháy

Máy tính nhúng Jetson Nano được kết nối với màn hình đã hiển thị được hình ảnh gửi về từ camera, trong đó, hình ảnh từ camera đã được tính toán qua mô hình trí tuệ nhân tạo để đánh giá nguy cơ cháy, sau khi đã xác định được đó là nguy cơ cháy, mô

hình trí tuệ nhân tạo đã khoanh vùng và hiển thị lên trên màn hình được kết nối với Jetson Nano.

Trên đây là kết quả thực nghiệm về nhận dạng nguy cơ cháy trên robot 3 bánh sử dụng bánh xe Omni, có thể thấy máy tính nhúng Jetson Nano đã thực hiện được việc tính toán nguy cơ cháy từ mô hình trí tuệ nhân tạo, từ đó có thể giúp robot điều hướng di chuyển và đưa ra cảnh báo nguy cơ cháy. Về mô hình trí tuệ nhân tạo, việc thực nghiệm của nhóm đưa ra kết quả mô hình nhận dạng với tốc độ tốt, độ chính xác cao, từ đó giúp việc nhận dạng và cảnh báo đạt hiệu quả tốt trong việc phòng tránh nguy cơ hỏa hoạn.

KẾT LUẬN

Đánh giá kết quả đạt được

- *Xây dựng mô hình 3D và chế tạo robot sử dụng 3 bánh Omni.*
- *Tính toán bài toán động học và động lực học cho robot.*
- *Xây dựng thuật toán điều khiển và nhận dạng, đo khoảng cách đám cháy cho robot.*
- *Xây dựng bộ điều khiển quỹ đạo.*
- *Thực nghiệm cho robot di chuyển theo quỹ đạo thực tế, kiểm thử mô hình nhận dạng nguy cơ cháy bằng trí tuệ nhân tạo.*

Hạn chế trong quá trình nghiên cứu

- *Bộ điều khiển vẫn gây ra sai số nhỏ, cần lựa chọn thông số tối ưu hơn nữa để đảm bảo tính chính xác của robot.*
- *Cần xây dựng phương pháp triệt tiêu sai số tích lũy để giúp robot có thể hoạt động độc lập trong thời gian dài mà vẫn đảm bảo độ chính xác.*
- *Cải thiện mô hình nhận dạng để tăng thêm tốc độ nhận dạng.*

Hướng phát triển

Hoàn thiện hơn nữa mô hình điều khiển "robot nhận dạng và cảnh báo sớm đám cháy ứng dụng trí tuệ nhân tạo" vì đó là cơ sở để nâng cao tính ứng dụng và những vấn đề tự hành của robot trong tương lai.

Song song với việc hoàn thiện mô hình điều khiển là việc gia tăng công việc thực nghiệm để có thể đưa ra nhiều kết quả, dựa trên kết quả thực nghiệm để hoàn thiện khả năng linh động và tính chính xác của robot. Nâng cao độ tin cậy của mô hình nhận dạng nguy cơ cháy ứng dụng trí tuệ nhân tạo cũng nâng cao tính ứng dụng của robot.

Như vậy, tuy còn nhiều vấn đề cần cải thiện và phát triển nhưng mô hình "robot nhận dạng và cảnh báo sớm nguy cơ cháy" đã hoàn thành được những nhiệm vụ cơ bản đề ra, hướng phát triển nhằm đưa mô hình robot nâng cao tính chính xác và ứng dụng cao trong thực tế, giúp con người giảm thiểu tối đa nguy cơ hỏa hoạn.

PHỤ LỤC

(Mã nguồn nhận dạng đám cháy ứng dụng trí tuệ nhân tạo)

```
import argparse
import os
import platform
import sys
from pathlib import Path

import torch
import torch.backends.cudnn as cudnn

FILE = Path(__file__).resolve()
ROOT = FILE.parents[0]  # YOLOv5 root directory
if str(ROOT) not in sys.path:
    sys.path.append(str(ROOT))  # add ROOT to PATH
ROOT = Path(os.path.relpath(ROOT, Path.cwd()))  # relative

from models.common import DetectMultiBackend
from utils.dataloaders import IMG_FORMATS, VID_FORMATS,
LoadImages, LoadStreams
from utils.general import (LOGGER, check_file,
check_img_size, check_imshow, check_requirements,
colorstr, cv2,
                           increment_path,
non_max_suppression, print_args, scale_coords,
strip_optimizer, xyxy2xywh)
from utils.plots import Annotator, colors, save_one_box
from utils.torch_utils import select_device,
smart_inference_mode, time_sync



@smart_inference_mode()
def run(
        weights=ROOT / 'yolov5s.pt',  # model.pt path(s)
```

```
        source=ROOT / 'data/images',  # file/dir/URL/glob,
0 for webcam
        data=ROOT / 'data/somke.yaml',  # dataset.yaml path
        imgsz=(640, 640),  # inference size (height, width)
        conf_thres=0.25,  # confidence threshold
        iou_thres=0.45,  # NMS IOU threshold
        max_det=1000,  # maximum detections per image
        device='',  # cuda device, i.e. 0 or 0,1,2,3 or cpu
        view_img=False,  # show results
        save_txt=False,  # save results to *.txt
        save_conf=False,  # save confidences in --save-txt
labels
        save_crop=False,  # save cropped prediction boxes
        nosave=False,  # do not save images/videos
        classes=None,  # filter by class: --class 0, or --
class 0 2 3
        agnostic_nms=False,  # class-agnostic NMS
        augment=False,  # augmented inference
        visualize=False,  # visualize features
        update=False,  # update all models
        project=ROOT / 'runs/detect',  # save results to
project/name
        name='exp',  # save results to project/name
        exist_ok=False,  # existing project/name ok, do not
increment
        line_thickness=3,     #  bounding  box  thickness
(pixels)
        hide_labels=False,  # hide labels
        hide_conf=False,  # hide confidences
        half=False,  # use FP16 half-precision inference
        dnn=False,  # use OpenCV DNN for ONNX inference
):
    source = str(source)
    save_img = not nosave and not source.endswith('.txt')
# save inference images
```

```
    is_file = Path(source).suffix[1:] in (IMG_FORMATS +
VID_FORMATS)
    is_url = source.lower().startswith(('rtsp://',
'rtmp://', 'http://', 'https://'))
    webcam = source.isnumeric() or source.endswith('.txt')
or (is_url and not is_file)
    if is_url and is_file:
        source = check_file(source)  # download

    # Directories
    save_dir = increment_path(Path(project) / name,
exist_ok=exist_ok)  # increment run
    (save_dir / 'labels' if save_txt else
save_dir).mkdir(parents=True, exist_ok=True)  # make dir

    # Load model
    device = select_device(device)
    model = DetectMultiBackend(weights, device=device,
dnn=dnn, data=data, fp16=half)
    stride, names, pt = model.stride, model.names, model.pt
    imgsz = check_img_size(imgsz, s=stride)  # check image
size

    # Dataloader
    if webcam:
        view_img = check_imshow()
        cudnn.benchmark = True  # set True to speed up
constant image size inference
        dataset = LoadStreams(source, img_size=imgsz,
stride=stride, auto=pt)
        bs = len(dataset)  # batch_size
    else:
        dataset = LoadImages(source, img_size=imgsz,
stride=stride, auto=pt)
        bs = 1  # batch_size
    vid_path, vid_writer = [None] * bs, [None] * bs
```

```
    # Run inference
    model.warmup(imgsz=(1 if pt else bs, 3, *imgsz))  # warmup
    seen, windows, dt = 0, [], [0.0, 0.0, 0.0]
    for path, im, im0s, vid_cap, s in dataset:
        t1 = time_sync()
        im = torch.from_numpy(im).to(device)
        im = im.half() if model.fp16 else im.float()  # uint8 to fp16/32
        im /= 255  # 0 - 255 to 0.0 - 1.0
        if len(im.shape) == 3:
            im = im[None]  # expand for batch dim
        t2 = time_sync()
        dt[0] += t2 - t1

        # Inference
        visualize = increment_path(save_dir / Path(path).stem, mkdir=True) if visualize else False
        pred = model(im, augment=augment, visualize=visualize)
        t3 = time_sync()
        dt[1] += t3 - t2

        # NMS
        pred = non_max_suppression(pred, conf_thres, iou_thres, classes, agnostic_nms, max_det=max_det)
        dt[2] += time_sync() - t3

        # Second-stage classifier (optional)
        # pred = utils.general.apply_classifier(pred, classifier_model, im, im0s)

        # Process predictions
        for i, det in enumerate(pred):  # per image
```

```
            seen += 1
            if webcam:  # batch_size >= 1
                p, im0, frame = path[i], im0s[i].copy(),
dataset.count
                s += f'{i}: '
            else:
                p, im0, frame = path, im0s.copy(),
getattr(dataset, 'frame', 0)

            p = Path(p)  # to Path
            save_path = str(save_dir / p.name)  # im.jpg
            txt_path = str(save_dir / 'labels' / p.stem) +
('' if dataset.mode == 'image' else f'_{frame}')  # im.txt
            s += '%gx%g ' % im.shape[2:]  # print string
            gn = torch.tensor(im0.shape)[[1, 0, 1, 0]]  #
normalization gain whwh
            imc = im0.copy() if save_crop else im0  # for
save_crop
            annotator = Annotator(im0,
line_width=line_thickness, example=str(names))
            if len(det):
                # Rescale boxes from img_size to im0 size
                det[:, :4] = scale_coords(im.shape[2:],
det[:, :4], im0.shape).round()

                # Print results
                for c in det[:, -1].unique():
                    n = (det[:, -1] == c).sum()    #
detections per class
                    s += f"{n} {names[int(c)]}{'s' * (n >
1)}, "  # add to string

                # Write results
                for *xyxy, conf, cls in reversed(det):
                    if save_txt:  # Write to file
```

```
                        xywh                                    =
(xyxy2xywh(torch.tensor(xyxy).view(1,   4))   /   gn).view(-
1).tolist()  # normalized xywh
                        line   =   (cls,   *xywh,   conf)   if
save_conf else (cls, *xywh)  # label format
                        with  open(f'{txt_path}.txt',  'a')
as f:
                            f.write(('%g        '        *
len(line)).rstrip() % line + '\n')


                    if save_img or save_crop or view_img:
# Add bbox to image
                        c = int(cls)  # integer class
                        label  =  None  if  hide_labels  else
(names[c] if hide_conf else f'{names[c]} {conf:.2f}')
                        annotator.box_label(xyxy,   label,
color=colors(c, True))
                    if save_crop:
                        save_one_box(xyxy,             imc,
file=save_dir  /  'crops'  /  names[c]  /  f'{p.stem}.jpg',
BGR=True)


            # Stream results
            im0 = annotator.result()
            if view_img:
                if platform.system() == 'Linux' and p not
in windows:
                    windows.append(p)
                    cv2.namedWindow(str(p),
cv2.WINDOW_NORMAL | cv2.WINDOW_KEEPRATIO)  # allow window
resize (Linux)
                    cv2.resizeWindow(str(p), im0.shape[1],
im0.shape[0])
                cv2.imshow(str(p), im0)
                cv2.waitKey(1)  # 1 millisecond


            # Save results (image with detections)
```

```
            if save_img:
                if dataset.mode == 'image':
                    cv2.imwrite(save_path, im0)
                else:  # 'video' or 'stream'
                    if vid_path[i] != save_path:  # new
video
                        vid_path[i] = save_path
                        if isinstance(vid_writer[i],
cv2.VideoWriter):
                            vid_writer[i].release()      #
release previous video writer
                        if vid_cap:  # video
                            fps                          =
vid_cap.get(cv2.CAP_PROP_FPS)
                            w                            =
int(vid_cap.get(cv2.CAP_PROP_FRAME_WIDTH))
                            h                            =
int(vid_cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
                        else:  # stream
                            fps, w, h = 30, im0.shape[1],
im0.shape[0]
                        save_path                        =
str(Path(save_path).with_suffix('.mp4'))  # force *.mp4
suffix on results videos
                        vid_writer[i]                    =
cv2.VideoWriter(save_path,
cv2.VideoWriter_fourcc(*'mp4v'), fps, (w, h))
                    vid_writer[i].write(im0)


        # Print time (inference-only)
        LOGGER.info(f'{s}Done. ({t3 - t2:.3f}s)')


    # Print results
    t = tuple(x / seen * 1E3 for x in dt)  # speeds per
image
```

```
    LOGGER.info(f'Speed:   %.1fms   pre-process,   %.1fms
inference, %.1fms NMS per image at shape {(1, 3, *imgsz)}'
% t)
    if save_txt or save_img:
        s = f"\n{len(list(save_dir.glob('labels/*.txt')))}
labels saved to {save_dir / 'labels'}" if save_txt else ''
        LOGGER.info(f"Results  saved  to  {colorstr('bold',
save_dir)}{s}")
    if update:
        strip_optimizer(weights[0])  # update model (to fix
SourceChangeWarning)



def parse_opt():
    parser = argparse.ArgumentParser()
    parser.add_argument('--weights', nargs='+', type=str,
default=ROOT      /      'runs/train/exp/weights/last.pt',
help='model path(s)')
    parser.add_argument('--source', type=str, default=ROOT
/ 'data/images', help='file/dir/URL/glob, 0 for webcam')
    parser.add_argument('--data', type=str, default=ROOT /
'data/smoke.yaml', help='(optional) dataset.yaml path')
    parser.add_argument('--imgsz', '--img', '--img-size',
nargs='+', type=int, default=[640], help='inference size
h,w')
    parser.add_argument('--conf-thres',        type=float,
default=0.25, help='confidence threshold')
    parser.add_argument('--iou-thres',         type=float,
default=0.45, help='NMS IoU threshold')
    parser.add_argument('--max-det',             type=int,
default=1000, help='maximum detections per image')
    parser.add_argument('--device', default='', help='cuda
device, i.e. 0 or 0,1,2,3 or cpu')
    parser.add_argument('--view-img', action='store_true',
help='show results')
    parser.add_argument('--save-txt', action='store_true',
help='save results to *.txt')
```

```
    parser.add_argument('--save-conf',
action='store_true', help='save confidences in --save-txt
labels')
    parser.add_argument('--save-crop',
action='store_true', help='save cropped prediction boxes')
    parser.add_argument('--nosave',   action='store_true',
help='do not save images/videos')
    parser.add_argument('--classes', nargs='+', type=int,
help='filter by class: --classes 0, or --classes 0 2 3')
    parser.add_argument('--agnostic-nms',
action='store_true', help='class-agnostic NMS')
    parser.add_argument('--augment',  action='store_true',
help='augmented inference')
    parser.add_argument('--visualize',
action='store_true', help='visualize features')
    parser.add_argument('--update',   action='store_true',
help='update all models')
    parser.add_argument('--project',    default=ROOT    /
'runs/detect', help='save results to project/name')
    parser.add_argument('--name',           default='exp',
help='save results to project/name')
    parser.add_argument('--exist-ok', action='store_true',
help='existing project/name ok, do not increment')
    parser.add_argument('--line-thickness',     default=3,
type=int, help='bounding box thickness (pixels)')
    parser.add_argument('--hide-labels',    default=False,
action='store_true', help='hide labels')
    parser.add_argument('--hide-conf',      default=False,
action='store_true', help='hide confidences')
    parser.add_argument('--half',     action='store_true',
help='use FP16 half-precision inference')
    parser.add_argument('--dnn',      action='store_true',
help='use OpenCV DNN for ONNX inference')
    opt = parser.parse_args()
    opt.imgsz *= 2 if len(opt.imgsz) == 1 else 1  # expand
    print_args(vars(opt))
    return opt
```

```
def main(opt):
    check_requirements(exclude=('tensorboard', 'thop'))
    run(**vars(opt))


if __name__ == "__main__":
    opt = parse_opt()
    main(opt)
```

(Mã nguồn hiệu chỉnh camera)

```
import sys
import numpy as np
import time
import imutils
import cv2

def undistorted(frameR, frameL):

    # Load parameters attained from "calib_parameters.py"
    ret = np.load("./Calibration/camera_params/ret.npy")
    K = np.load("./Calibration/camera_params/K.npy")
    dist = np.load("./Calibration/camera_params/dist.npy")
    rvecs = np.load("./Calibration/camera_params/rvecs.npy")
    tvecs = np.load("./Calibration/camera_params/tvecs.npy")


    hR,wR = frameR.shape[:2]
    hL,wL = frameL.shape[:2]
```

```
    new_camera_matrixR, roiR = cv2.getOptimalNewCameraMatrix(K,dist,(wR,hR),1,(wR,hR))
    new_camera_matrixL, roiL = cv2.getOptimalNewCameraMatrix(K,dist,(wL,hL),1,(wL,hL))

    #Undistort images
    frame_undistortedR = cv2.undistort(frameR, K, dist, None, new_camera_matrixR)
    frame_undistortedL = cv2.undistort(frameL, K, dist, None, new_camera_matrixL)

    ##Uncomment if you want help lines:
    #frame_undistortedR = cv2.line(frame_undistortedR, (0,int(hR/2)), (wR,240), (0, 255, 0) , 5)
    #frame_undistortedR = cv2.line(frame_undistortedR, (int(wR/2),0), (int(wR/2),hR), (0, 255, 0) , 5)
    #frame_undistortedL = cv2.line(frame_undistortedL, (int(wL/2),0), (int(wL/2),hL), (0, 255, 0) , 5)
    #frame_undistortedL = cv2.line(frame_undistortedL, (0,int(hL/2)), (wL,240), (0, 255, 0) , 5)
    print(K)

    return frame_undistortedR, frame_undistortedL
```

TÀI LIỆU THAM KHẢO

Tiếng Anh

[1] H. Saygin SUCUOGLU, Ismail BOGREKCI, Pinar DEMIRCIOGLU. Development of Mobile Robot with Sensor Fusion Fire Detection Unit. Adnan Menderes University, Aydin, 09010, Turkey.

[2] Sridevi Chitti, P. Ramchandar Rao, Ch. Padmaja and D Raghava Kumari. Fire Detection and Direction Control of Fire Fighting Robot.

[3] Roland Siegwart, Illah R. Noubakhsh. Introduction to Autonomous Mobile Robots. Library of Congress Cataloging-in-Publication Data

[4] Jos´e Gonc¸alves, Jos´e Lima, H´elder Oliveira and Paulo Costa. Sensor and actuator modeling of a realistic wheeled mobile robot simulator. Faculdade de Engenharia da Universidade do Porto, DEEC Porto, Portugal

[5] Yasir Dawood Salman, Ku Ruhana Ku-Mahamud and Eiji Kamioka. Distance measurement for self-driving cars using stereo camera, Proceedings of the 6th International Conference on Computing and Informatics, ICOCI 2017 25-27April, 2017 Kuala Lumpur.

Website

[6] https://viblo.asia/p/tong-hop-kien-thuc-tu-yolov1-den-yolov5-phan-3-63vKjgJ6Z2R

[7] https://www.dientuhello.com/do-toc-do-dong-co-dung-encoder-voi-arduino/

[8] https://vietmachine.com.vn/dieu-khien-arduino-dc-l298n-pwm-mach-cau-h.html